

**REAL-TIME OBJECT DETECTION FOR AUTONOMOUS
VEHICLES USING DEEP LEARNING: BALANCING
ACCURACY, SPEED, AND ROBUSTNESS IN DYNAMIC
ENVIRONMENTS**

Abstract

The project titled Real-Time Object Detection for Autonomous Vehicles Using Deep Learning: trade-offs inspired by the necessity of yielding the best possible results in terms of the distances the cars must cover with such safeguards. The paper Balanced Accuracy, Speed, and Robustness in Dynamic Environments examines the use of deep learning to enhance object detection technologies for use in self-driving cars. Self-driving cars need refined object recognition technologies, which make operation safe in unforeseeable circumstances. However, such a combination of speed, accuracy, and robustness is not easy to achieve, which poses great difficulties. Advanced image detection algorithms that are used in this work include Faster R-CNN, YOLO, and SSD to improve the detection in different environments. All the models are evaluated based on occlusion, small object, and lighting and weather condition. It is in the category of transfer learning that improvements in the generalization of the models and the computational advantages are realized. Moreover, tuning methods for hyperparameters and data augmentation improve accuracy in real-time applications and optimal consumption of hardware resources. The findings show that localization precision and feature detection speed go hand in hand as Faster R-CNN is the most accurate while YOLO, and SSD are less accurate but yield results much faster. Such algorithms bring insights to design more efficient real-time object detection systems used to enhance safety in self-driving cars.

Table of Contents

Abstract.....	2
Chapter 1: Introduction.....	7
1.1 Background.....	7
1.2 Problem Statement.....	7
1.3 Rationale.....	8
1.4 Research Aims and Objectives.....	8
1.5 Research Questions.....	8
1.6 Significance of the Study.....	9
1.7 Structure of the Dissertation.....	9
1.8 Summary.....	10
Chapter 2: Literature Review.....	10
2.1 Introduction.....	10
2.2 Overview of object detection's role in autonomous vehicles.....	10
2.3 Challenges in Real-Time Object Detection.....	11
2.4 Advancements in Deep Learning for Object Detection.....	12
2.5 Robustness in Object Detection Models.....	13
2.6 Efficiency and Real-Time Performance.....	14
2.7 Comparative Analysis of State-of-the-Art Algorithms.....	15
2.9 Theoretical Framework.....	16
2.10 Literature Gap.....	16
2.11 Summary.....	17
Chapter 3 Methodology.....	17
3.1 Introduction to Methodology.....	17
3.2 Overview of Selected Object Detection Algorithms.....	17

3.2.1 Justification for Using Multiple Algorithms.....	21
3.3 Dataset Preprocessing and Preparation.....	21
3.3.1 Importance of Accurate Image Annotation in Object Detection.....	21
3.3.2 Data Annotation.....	22
3.3.3 Train-Test Split.....	22
3.3.4 Importance of These Steps.....	23
3.4 Implementation of Object Detection Models.....	23
3.4.1 Frameworks Used.....	23
3.4.2 Transfer Learning.....	23
3.4.2 Training Procedure.....	24
3.4.3 Loss Functions.....	24
3.4.4 Justification of Model Parameters and Frameworks.....	25
3.5 Model Evaluation and Comparison Metrics.....	25
3.6 Hyperparameter Tuning and Model Optimization.....	26
3.7 Conclusion.....	26
Chapter 4: Result.....	27
4.1 Overview of Model Performance.....	27
4.2 Training Process and Loss Analysis.....	28
4.3. Evaluation Metrics.....	30
4.4. Comparison of Model Results.....	36
Comparison table of models.....	39
4.5. Visual Results: Bounding Box Predictions.....	42
4.6. Statistical Significance of Results.....	43
4.7. Summary of Findings.....	44
Chapter 5: Discussion.....	45

5.1. Model Performance Insights.....	45
5.2. Effectiveness of Transfer Learning.....	46
5.3. Impact of Data Annotation and Preprocessing.....	47
5.4. Evaluation of Model Robustness.....	47
5.5. Comparison with Previous Research.....	48
5.6 Challenges and Limitations.....	49
5.7. Practical Implications.....	49
Chapter 6: Conclusion.....	50
6.1 Summary of Contributions.....	50
6.2 Implications for Future Research.....	51
6.3. Ethical Considerations.....	52
6.4 Final Reflection.....	52
References.....	54
Appendix 1.....	62
Appendix 2.....	76

Table of Figures

Figure 1 Enhancing Object Detection in Self-Driving Cars.....	10
Figure 2 Pothole detection for autonomous vehicles using deep learning.....	11
Figure 3 Overview of Selected Object Detection Algorithms.....	20
Figure 4 Dataset Preprocessing and Preparation.....	21
Figure 5 Implementation of Object Detection Models.....	23
Figure 6 Image Detection (1).....	26
Figure 7 Image Detection (2).....	27
Figure 8 Image Detection (3).....	27
Figure 9 Image Detection (4).....	28
Figure 10 Image Detection (5).....	29
Figure 11 Image Detection (6).....	29
Figure 12 Image Detection (7).....	30
Figure 13 Image Detection (8).....	31
Figure 14 Image Detection (9).....	32
Figure 15 Image Detection (10).....	33
Figure 16 Image Detection (11).....	34
Figure 17 Image Detection (12).....	35
Figure 18 Image Detection (13).....	36
Figure 19 Image Detection (14).....	37
Figure 20 Image Detection (15).....	38
Figure 21 Image Detection (16).....	39
Figure 22 Image Detection (17).....	40
Figure 23 Image Detection (18).....	41
Figure 24 Image Detection (19).....	42
Figure 25 Image Detection (20).....	43

Chapter 1: Introduction

1.1 Background

Autonomous Vehicles (AVs) represent an immense amount of development inside the transportation industry (Bathla *et al.*, 2022). It is going to transform the way we commute every day with enhanced safety, efficiency, and convenience. At the heart of any self-driving vehicle lies a sophisticated sensor and algorithm suite that perceives the environment and decides in real-time. Other technologies, like object detection, become very important because they help AVs to be aware of different objects on the road be it pedestrians, other vehicles, or obstructions (Sharma and Rana, 2024). This must be done with lots of accuracy and speed, as a minute delay or error in judgment can lead to a huge disaster. Especially, object detection in a real-time dynamic atmosphere is going to be tricky. These challenges emerge from the very high speed of processing huge volumes of data, combined with high accuracy levels. Detection is further complexed by these environmental changes, illumination conditions, weather changes, and indeed small or occluded objects.

Moreover, due to limited computational resources with AV platforms, highly efficient algorithms without any loss of performance were necessitated. Deep learning served as a powerful tool in the advancement of technologies used for object detection in AVs (Chen *et al.*, 2020). Learning accurate classification using large datasets and complex neural networks allows for wide possibilities; however, this stays as a pertinent challenge with regard to the balance of accuracy, speed, and robustness. This dissertation seeks to address these predicaments by developing and fine-tuning deep learning-based models for real-time object detection's role in autonomous vehicles; this guarantees their serviceability in an extended range of conditions.

1.2 Problem Statement

The real-time object detection in AVs has imminent challenges and may require a fine balance of model accuracy, speed, and robustness (Pavel *et al.*, 2022). Maximum accuracy in detection is crucial, but at most instances, this is achieved at the expense of speed, among many factors that are important for real-time processing. The AVs are supposed to perform rightly in very diverse and dynamical environments, wherein instances include lighting variations, weather, small/occluded object presence, etc., that are expected to dramatically affect detection performance. Only to make matters worse, the problems of these detection approaches are

compounded by the fact that most AV platforms have limited hardware resources (Giannaros *et al.*, 2023). These thereby substantiate the dire need to frame deep learning models that can meet all such demands without compromise in their performance terms. This project therefore targets developing and optimizing robust deep learning-based object detection models within proper constraints to ensure robust real-time performance in very complex driving scenarios.

1.3 Rationale

To ensure that autonomous vehicles can navigate safely and effectively, the rapid development of the same calls for extremely efficient and very reliable object-detection-based systems. However, the balance between accuracy, speed, and at the same time robustness is not easy, particularly for very hard and dynamic scenes, which is its actual demand in state-of-the-art models. Motivated by the objective of enhancing deep model performance in real-time object detection, this work makes them able to execute different real-time tasks on available hardware resources enabled with AVs. Addressing these challenges, this paper aims at contributing toward the development of safer and more reliable driverless technology.

1.4 Research Aims and Objectives

The primary aim of this research is to develop and optimize deep learning-based models for real-time object detection in autonomous vehicles. The objectives are:

- To design and optimize deep learning models for real-time object detection in autonomous vehicles
- To enhance the robustness of object detection models under varying environmental conditions
- To improve detection performance for small and occluded objects within real-world driving scenarios
- To develop computationally efficient models that can perform real-time object detection on the limited hardware resources available in autonomous vehicles.

1.5 Research Questions

- How can deep learning models be designed and optimized to balance accuracy and real-time performance for object detection in autonomous vehicles?
- What methodologies can be employed to enhance the robustness of object detection models under varying environmental conditions encountered by autonomous vehicles?

- What techniques can improve the accuracy of detecting small and occluded objects in autonomous driving scenarios?
- How can computational efficiency be achieved in deep learning models for real-time object detection without compromising detection performance?

1.6 Significance of the Study

This is tremendously important work toward enabling autonomous driving, as it specifically pinpoints the important reasons for challenges in the real-time object detection exercise a core component of the safe functioning of AVs. Better and faster object detection in diverse environments would support better decision-making, thus safe navigation (Yang *et al.*, 2023). Developing and fine-tuning deep learning models to handle accuracy, speed, and robustness could be a very vital part of research in improving the reliability of AV systems, given that such failures are related to most accidents. The results of the study will be important for many areas of deep learning, starting with object detection, through new insights into model optimization for real-time performance over limited hardware resources. More so, this study shall yield practical solutions that could be implemented in real-world situations while filling extant lacunae in the literature by addressing specific issues of detection for small and occluded objects in diverse environmental conditions. Ultimately, this study will support the development of safe, autonomous vehicle devices, i.e. resilient, and efficient AV systems, able to operate in an environment characterized by complexity and uncertainty.

1.7 Structure of the Dissertation

This dissertation comprises six main chapters. This chapter 2 will contain a literature review that finds the researches existing for the detection of objects using deep learning and applications in autonomous vehicles. Chapter 3 addresses the details from the research methodology with respect to the design process, optimization, and evaluation for the development of the proposed models. Chapter 4 includes results and a deep analysis related to model performance. Chapter 5 of the thesis discussion and Chapter 6 contains has reached conclusions and recommendations based on the findings of the research and suggestions for future research. All the chapters have the research aims and objectives matched and also contribute to the overall aim of actually enhancing the detection of real-time objects detected by the AVs.

1.8 Summary

The introduction has followed up with the main problems in real-time object detection for self-driving cars, particularly the need for balancing accuracy, speed, and robustness under dynamic scenarios. The same also articulates the issue of the vital role of computational efficiency and the challenge of small and occluded object detection. The possibility of highlighting research aims, objectives, and research questions establishes the question of the importance of carrying out the study in the developments of autonomous driving technology. Dealing with these problems is key in realizing safe and reliable autonomous vehicles; hence, research that addresses these issues is both applicable in the current status of technology and urgent in satisfactory living.

Chapter 2: Literature Review

2.1 Introduction

The rapid development of autonomous vehicles has put a strong emphasis on the need for advanced object detection systems that are able to perform, particularly in environments with a large variety of scenarios, in real time. Object detection is a significant attribute of AV technology to detect and differentiate objects around vehicles for proper navigation. This chapter presents the state-of-the-art review of the literature on real-time object detection in AVs; therein special attention is focused on challenges and advancements in deep learning models. The review has also attempted to portray accuracy, speed, and robustness trade-offs, state the state-of-the-art research, and identify research gaps that the current dissertation can contribute to.

2.2 Overview of object detection's role in autonomous vehicles

According to Parekh *et al.*, 2022, object detection is a crucial function of any AV to facilitate safe and effective navigation decisions by the vehicle. They identify and classify objects such as pedestrians, vehicles, or obstacles with a high level of accuracy. AVs can thus perceive their environment and react accordingly. In view of this, real-time object detection becomes even more important when the environment is dynamic and the conditions are expected to change fast. Thereby, the capability of real-time processing ensures that a response from an AV will come to some kind of unexpected situation or fast-changing slight obstacles standing around it in the way or in the traffic pattern, not necessarily leading to an accident.

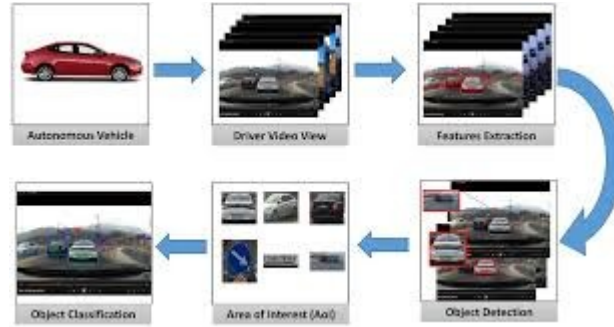


Figure 1 Enhancing Object Detection in Self-Driving Cars

The impact of deep learning on the performance of object detection systems in AVs is dramatic (Bathla *et al.*, 2022). Traditional methods, usually designed based on hand-crafted features and rule-based algorithms, often couldn't bear the complexities and variability encountered in real-world driving scenarios. Through this, deep learning particularly convolutional neural networks changed each paradigm approach that was possible in object detection. This resulted in great accuracy and robustness improvements in detection, even in hard conditions. However, these gains must be balanced with the need for and computational efficiency. Therefore, ensuring the obtained improvements are without delays is a critical challenge. This dissertation hence tries all the possible ways to fix the hitch of optimally compressing deep learning models that can be put into real-time applications in AVs.

2.3 Challenges in Real-Time Object Detection

Real-time object perception in an autonomous vehicle (AV) processing system needs to strike the right balance between accuracy and speed (Shahian Jahromi *et al.*, 2019). Correct object recognition and classification largely depend on high accuracy. The highly accurate system will certainly bring computational cost that is on the higher side and most probably will decrease the processing speed. On the other hand, the models that are fast may be impaired with less accuracy, and then greater risks would result in such quickly changing driving environments. This trade-off is crucial for developing object detection AV systems where both precision and real-time execution are crucial.

Real-time object detection is further complicated by these properties' robustness in diverse environmental conditions. As pointed out further by Hoffmann *et al.*, 2020 and Muhammad *et al.*, 2020, this shows that AVs have to work reliably across a vast range and variety of different scenarios, including those with varying lighting, harsh weather, and highly dynamic environments.

The aim is to have robust detection models that are invariant to maintaining high performance despite such variations. Please, keep in mind that one of the main objectives, including occluded object detection, is small size objects, and in common, all that is a very regular situation on roads. The models for detection, mentioned by Cao *et al.*, 2021 and Mahaur *et al.*, 2022, can easily miss small objects, while occlusions may contribute to hiding some parts of an object from the field of view, complicating its accurate detection. The solution given to the problem by these two is most important in the development of dependable and efficient object detection in AVs.

2.4 Advancements in Deep Learning for Object Detection

As reported by the Gupta *et al.*, 2021, the advances in the evolution of deep learning models have greatly changed the landscape of object detection related to autonomous vehicles. Classic object detection methods relied on extensive manual feature extraction and the use of classical machine learning algorithms, hence most of the time they faced big challenges to withstand the complexity of the real-world environments. The development of convolutional neural networks brought a shift in this paradigm: With them, there was automated feature extraction directly from data, which highly increased detection accuracy and robustness. In their work, Zhao *et al.*, 2019 reviewed advances in the deep learning evolution and traced them from the core idea in first models, such as R-CNN, to more sophisticated architectures such as Faster R-CNN, YOLO, and SSD, which, in turn, became the backbone of recent object detection systems.

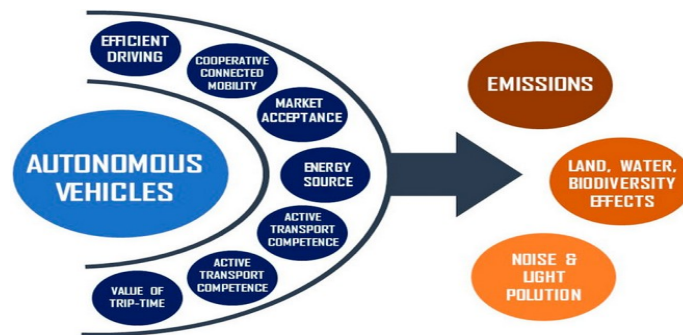


Figure 2 Pothole detection for autonomous vehicles using deep learning

Contrary to the development of autonomous deep learning models, a great volume of work has been carried out in the domain of developing hybrid frameworks integrating object detection with recognition tasks. This would therefore form a hybrid framework by Li *et al.*, 2020, which is based on deep learning, where the processes of detection and recognition are rolled into one underneath architecture. This provides the classification of objects detected, hence enabling the system not

only with the potential to enhance the complex scenarios of the driving environment but also to benefit from the improved classification performance of hybrid frameworks. Such frameworks tend to be more valuable in AVs, where a real-time decision calls for detection and most accurate identification of objects. Another critical area of development is generalization of object detection models to unseen environments.

Khosravian *et al.*, 2021 highlighted the need for developing methods that could perform highly even in conditions unseen during training. This generalization ability is critical for AVs, which have to work reliably amid all diverse and unforeseeable real-world scenarios. Domain adaptation and transfer learning have been explored as ways to improve model generalization enabling AVs to accurately detect objects even in novel or challenging settings. The improvements in object detection systems are driven by these advancements in deep learning there is a continued evolution in object detection that advances the performance of AVs with respect to both safety and reliability.

2.5 Robustness in Object Detection Models

Robustness is going to be the most crucial aspect considered for the object detection model in autonomous vehicles because the identified objects need to be recognized appropriately under highly diverse and more often unpredictable environmental circumstances. It will be of immense value in creating models that hold up under lighting, weather, and other dynamic factor changes for ensuring the safety and reliability of AVs. Song *et al.*, 2024 and Alaba & Ball 2023 described some strategies to increase the robustness of the object detection model. These include data **annotation**-based strategies to ensure the models are trained optimally under a wide environmental variation and the development of adaptive models to sensitively adjust to the real-time changes in the environment. An innovative way to improve robustness is captured in techniques of the dual-modality pertaining to image quality assessment, as considered by Geng *et al.*, 2022. This comes under the technological aspect of multi-sensor use or fusion of imaging modality vision and infrared for instance in enhancing the probability of detection in difficult circumstances, for example during low light or during a particular adverse weather conditions. Thus, in their fusion, variability of the environment can be effectively exploited to develop more accurate models for object detection.

Another powerful approach to enhancing robustness is the integration of deep learning-based sensor fusion within object detection models. Fayyad *et al.*, 2020 and Hassaballah *et al.*, 2020 echo

the same observation: data from cameras, LiDAR, radar, among others, necessitate fusion in order to realize perfect perception of the vehicle's surroundings. The deep learning models may account for single-sensor information limitations due to this sensor fusion approach for instance, a camera inaccuracy with regard to illumination or noise within the LiDAR data. By processing and fusing data from various sources, these models can achieve high robustness, assuring that the AVs under many different scenarios—will still be able to detect objects accurately. The improvements in robustness are crucial to develop AVs that can work in a safe and efficient manner across diverse environments. Robustness enhancement of object detection models is therefore expected to continue as a research focus, to proceed along waves of advancement in the AV technology by driving overall improvement in the performance and reliability of the AV systems.

2.6 Efficiency and Real-Time Performance

Models such as those for object detection in AVs have to consider computational efficiency. It stands to reason that a model must be designed to carry out difficult tasks in real time but still be both accurate and robust under the constraints of the low processing powers available on many AV platforms. Moreover, as noted by Wang *et al.*, 2019, this quality of computational efficiency is deemed appropriate because real-time processing requirements of autonomous vehicles need models to carry out detection operations at maximum wits and bounds, considering that their hardware resources are highly constrained. Several techniques have been developed in this area for efficient optimization of deep learning models in the domain of real-time performance applications. Key among them is model pruning, quantization, and the use of lightweight architectures such as MobileNet and SqueezeNet (Yang *et al.*, 2022). The computation workload is reduced, and maximum precision is retained. All these methods help in making the object detection models deployable on low-resource devices, and, therefore, are suitable for real-world AV.

Real-time object detection and tracking algorithms shall without doubt assist in making certain an AV can maneuver in a dynamic setting safely. The proposed real-time novel advanced novel algorithms for are discussed in Hoffmann *et al.*, 2020 and Staroverov *et al.*, These combine the process of detection and tracking in tracking the position and movement of objects around the vehicle on a continuous manner. The efficiency in the above processes makes these models very keen and fast to perform their autonomous driving hence being safe. Concerning computational

complexity and time response, which to a significant measure, thus becomes critical in making object detection models practical in AVs. That is why, if in the future the increase in the unique characteristics of AV technology will be solved in advanced ways, further innovations in these aspects will be required to guarantee the sufficiently reliable and safe operation of vehicles in complex and unpredictable conditions.

2.7 Comparative Analysis of State-of-the-Art Algorithms

The development of state-of-the-art deep learning algorithms in vehicle detection has improved the technology of autonomous vehicles. There have been comparative studies by, among others, Wang *et al.*, 2019 and Gengec *et al.*, 2021 on how effective these algorithms could be in practice. Wang *et al.*, 2019 have laid an overall comparison among many vehicle detection algorithms based on deep learning in terms of accuracy, speed, and computational efficiency. Their study thus comes to show high accurate algorithms, like Faster R-CNN and YOLO, normally come reduced from valuable computational resources, therefore not ideal for real-time application in resource-constrained environments. Other models, such as Single Shot MultiBox Detector (SSD), balance speed and accuracy in the detection process, able to turn it into real-time feasibility with AV applications. A line of development leading in that direction was the one introduced by Gengec *et al.*, 2021 to assess deep learning-based algorithms for detection ability in smart factory settings moderately related to the problems of real-time processing and robustness in very different conditions, partly posed by AVs. Their findings underscore the need for context-dependent optimizations it is worth noting that algorithms doing well on constrained laboratory conditions may not carry over to dynamic real-world scenarios.

Kuutti *et al.*, 2020 provided a more general review of current methodologies on a high level, analyzing their performance in relation to the various deployment scenarios that AVs find themselves in. Strong emphasis is laid by their work on the fact that no single algorithm universally 'outperforms' the rest but that rather each has strengths and weaknesses depending on the particular application and environmental conditions. Other examples include: suitable for large and/or very distinguishable objects, while others perform well towards small or occluded objects in cluttered scenes. Research and development would then need to continue fine-tuning these control algorithms and developing versatile models that can deliver reliable performance under a wide number of conditions. In conclusion, the comparative analysis of state-of-the-art algorithms acting

in vehicle detection emphasizes that the task is overwhelmingly complex. It also points to the continuously needed optimization of the models for specific use cases, in a way that balances the accuracy trade-off to meet real-world AV applications' demands.

2.9 Theoretical Framework

The theoretical underpinning of this research is based on the intersecting area of deep learning with real-time object detection within AVs. Broadly, the framework in question relies on some of the prime elements within computer vision and machine learning overall: current advances in CNNs and their adaption toward object-detection-related projects (Chen, 2024). The theoretical base is built up on the idea that these deep learning models can be implemented to be trained so they automatically learn and understand the features from larger datasets as this qualification enables them to quickly and accurately identify and classify the objects. The present framework is thereby centered on three major crucial focal points that are accuracy, speed, and robustness, which widely remain interdependent to each other with respect to the real-time object detection issue. Optimization theory can easily address these trade-offs between such factors in finding the best solution under predefined constraints be it computation resources or variability in environmental conditions (Kaim *et al.*, 2018). It will also incorporate theories such as sensor fusion and multimodal learning in synchronizing information from differing sources toward increasing detection robustness against different conditions that are noticeable very dynamically. Using these theoretical concepts, the development and optimization of object detection models that are accurate, fast, and resilient to real-world driving challenges in general, to contribute to the broad field of autonomous driving technologies are the focus of this study.

2.10 Literature Gap

Despite major successes of deep learning for object detection in autonomous vehicles, there still exist very recent and salient gaps with important significance. Much current literature has focused on the numerous trade-offs among accuracy, speed, and robustness. Very little work studies models that serve well in a diverse set of unseen circumstances. Another is the practical application of sensor fusion and multimodal learning to resource-constrained platforms in autonomous vehicles. Moreover, there is a lack of comprehensive studies in tackling real-time challenges for the detection of small and occluded objects, specifically for high-dynamic-driving-type scenarios.

This dissertation attends to these shortcomings by seeking to optimize deep learning models for improved generalization, robustness, and efficiency under real-world conditions.

2.11 Summary

This chapter has provided an overview of the state of recognition research using deep learning particularly for self-driving car application, giving consideration to both precision and velocity against resilience. Three approaches for change, theoretical support and for further development defines the progress between existing gaps in advancement.

Chapter 3 Methodology

3.1 Introduction to Methodology

This section briefly outlines the research approaches used in object detection in this work with a special focus on deep learning architectures namely; Region Convolutional Neural Network (R-CNN), Faster R-CNN, You Only Look Once (YOLO) and Single Shot MultiBox Detector (SSD). Such models were used since it was shown in the object detection that this approach performs well at the tasks, and the attained accuracy was nearly equal to the time needed for the computations. The present research therefore uses secondary research method because the data used in the present study is secondary. This is because the various models had to be chosen in such a way that they had to be compared with each other using the various characteristics like the accuracy, the mean average precision (mAP) and the speed of the detection. For example, the Faster R-CNN is said to give a better result of the objects in question, in comparison with other models such as YOLO, or SSD as used in real time processes because of the time and computational resources (Tan *et al.*, 2021). These models within the context of this study are used to find out what approach is suited for what object detection for the given data set used to tackle the research question (Srivastava *et al.*, 2020).

3.2 Overview of Selected Object Detection Algorithms

Object detection fictions are part of the application of computer vision and includes real application, self-driving car and monitoring use. In this study, four advanced algorithms are employed: These consist of R-CNN, Faster R-CNN with Region Proposal Network, the YOLO method and the SSD method. These algorithms also vary concerning how they balance the precision and quickness of their performance, which makes them applicable for use in several conditions.

R-CNN (Region-based Convolutional Neural Network)

R-CNN is a pioneering method for object detection that divides the process into two stages: first, to get the region proposals, and then secondly using a convolutional neural network to classify within these regions. Evaluating R-CNN, one can mention that even though its accuracy is higher compared to other approaches, it has some drawbacks will consume more time as each region proposal is forwarded to the CNN independently.

Classification Loss:

$$L_{\text{cls}}(p_i, u) = -\log(p_u)$$

Where p_i is the predicted probability of the class u (true class label), and $u \in \{0,1\}$ (background or object).

Bounding Box Regression Loss:

$$L_{\text{reg}}(t_i, v) = \sum_{i \in \{x, y, w, h\}} \text{smooth}_{L_1}(t_i - v_i)$$

Where t_i are the predicted bounding box coordinates and v_i are the ground truth coordinates.

Because of these limitations R-CNN is not preferred for real time applications but is used in applications where precision is of paramount importance.

Faster R-CNN

Faster R-CNN enhances R-CNN in that it incorporates the generation of the region proposals into the network with an RPN (Region Proposal Network). While this integration does away with the need for a separate step to propose regions, it has been found to enhance the speed whilst incurring minimal loss in accuracy.

RPN Loss (Region Proposal Network):

$$L_{\text{RPN}}(p_i, t_i) = \frac{1}{N_{\text{cls}}} \sum_i L_{\text{cls}}(p_i, p_i^*) + \lambda \frac{1}{N_{\text{reg}}} \sum_i [p_i^* L_{\text{reg}}(t_i, t_i^*)]$$

Where p_i is the predicted probability for region i , t_i are the predicted bounding box coordinates, p_i^* and t_i^* are the ground truth labels and coordinates, respectively, and λ is a balancing parameter.

Between the two methods of implementation of the R-CNN model, the faster has been deemed efficient for applications where accuracy is a necessity and at the same time, the speed could not be as fast as the applications that entail high real-time processing. It was found to have a strong performance in situations like autonomous driving and, small object detection with high accuracy, but the processing is slower than one-stage detectors like YOLO (Yang *et al.*, 2023).

YOLO (You Only Look Once)

YOLO introduces the object detection problem as a single regression problem therefore estimating both the bounding boxes as well as the class probabilities from the input image within a single computational iteration.

YOLO Loss Function:

$$L = L_{\text{coord}} + L_{\text{conf}} + L_{\text{class}}$$

Coordinate Loss:

$$L_{\text{coord}} = \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B 1_{ij}^{\text{obj}} [(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 + (\sqrt{w_i} - \sqrt{\hat{w}_i})^2 + (\sqrt{h_i} - \sqrt{\hat{h}_i})^2]$$

Confidence Loss:

$$L_{\text{conf}} = \sum_{i=0}^{S^2} \sum_{j=0}^B 1_{ij}^{\text{obj}} (C_i - \hat{C}_i)^2 + \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^B 1_{ij}^{\text{noobj}} (C_i - \hat{C}_i)^2$$

Class Loss:

$$L_{\text{class}} = \sum_{i=0}^{S^2} 1_i^{\text{obj}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2$$

This makes YOLO real-time hence suitable for applications where time is of the essence such as in self-driving cars. However, earlier versions of YOLO lose some amount of accuracy to speed, while YOLOv4, YOLOv3-Tiny is much better in this case (Kim *et al.*, 2020).

SSD (Single Shot Multibox Detector)

SSD is another single stage object detector similar to YOLO but with a different structure where features extracted from the image is divided into grids and several prediction for each cells of the grid are made.

SSD Loss Function:

$$L = L_{\text{conf}} + \alpha L_{\text{loc}}$$

Where L_{conf} is the confidence loss, and L_{loc} is the localization loss, with α being a weighting term.

Confidence Loss:

$$L_{\text{conf}} = - \sum_i \log(p_i^*)$$

Where p_i^* is the probability of the true class.

Localization Loss:

$$L_{\text{loc}} = \sum_{i=1}^N \sum_{m \in \{x, y, w, h\}} \text{smooth}_{L_1}(t_i^m - \hat{t}_i^m)$$

Where t_i^m and $\hat{t}_i^m$ are the predicted and ground truth box coordinates.

Although, SSD is slower than YOLO it performs better in some tasks especially in detecting the small objects. Because of its high responsiveness and adequate division of accuracy, SSD can be considered as one of the strong rivals in real-time object detection although the situation on the accuracy changes when it is used in complex conditions where it is less accurate in comparison with Faster R-CNN (Niranjan et al., 2021).

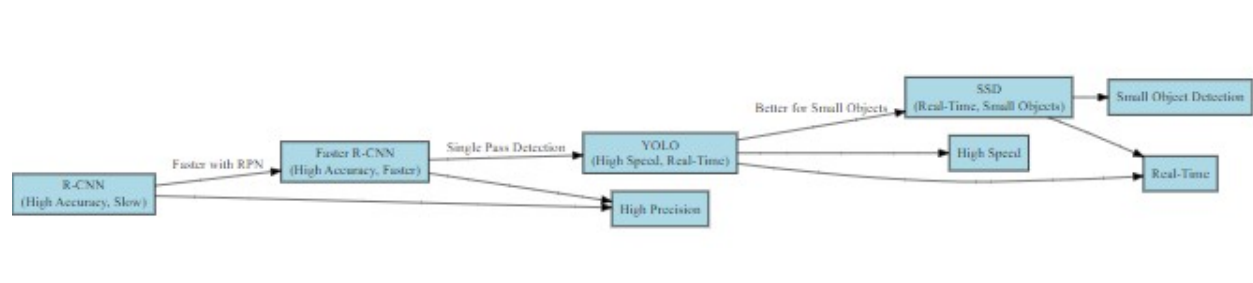


Figure 3 Overview of Selected Object Detection Algorithms

3.2.1 Justification for Using Multiple Algorithms

Through the comparative examination of these detection methods, there is a rich understanding of the effects of the trade-off between speed and accuracy made available by R-CNN, Faster R-CNN, YOLO, and SSD. With such a variety of algorithms, the study can determine the best functioning algorithm for specific aspects of object detection for example the detection of big or small objects or real time environments. Even though Faster R-CNN is precise, it is slower, which means that it is applied in high precision based tasks while YOLO's key attribute is speed and essential for real-time detection systems (Alkentar *et al.*, 2021).

3.3 Dataset Preprocessing and Preparation

As it wanted a dataset appropriate that is suitable for object detection, several steps of preprocessing were carried out, and they included data **annotation**, and train-test splitting. These steps were critical to fine-tune a model to perform better and improve its generalizing capabilities.

3.3.1 Importance of Accurate Image Annotation in Object Detection

This process is the labeling of each of the images used in the dataset by drawing rectangular shapes around the objects of interest followed by their class label. This is a very important stage in terms of object detection since the algorithms require high-quality tags in order to be able to recognize examples in other images. Correct **annotation** also helps the models increase the classification of the visual features with the right object categories thus improving detection (Chung *et al.*, 2020).

3.3.2 Data Annotation

The use of multiple transform operations including flip, rotate, scale and random crop was used in order to augment the data and minimize the problem of over fitting. These methods help add variation to the training data, by submitting samples that are different from the original data set, hence making the model to have the ability to learn better from new data sets that it has never encountered before. For instance, image rotation, flipping assists the model in learning the features that are invariant to image orientation as well as size. Here, by increasing the volume of data, the chance of the model fixing its attention on some particular patterns is eliminated; as it results in a high degree of model performance in different circumstances (Kaur *et al.*, 2021).

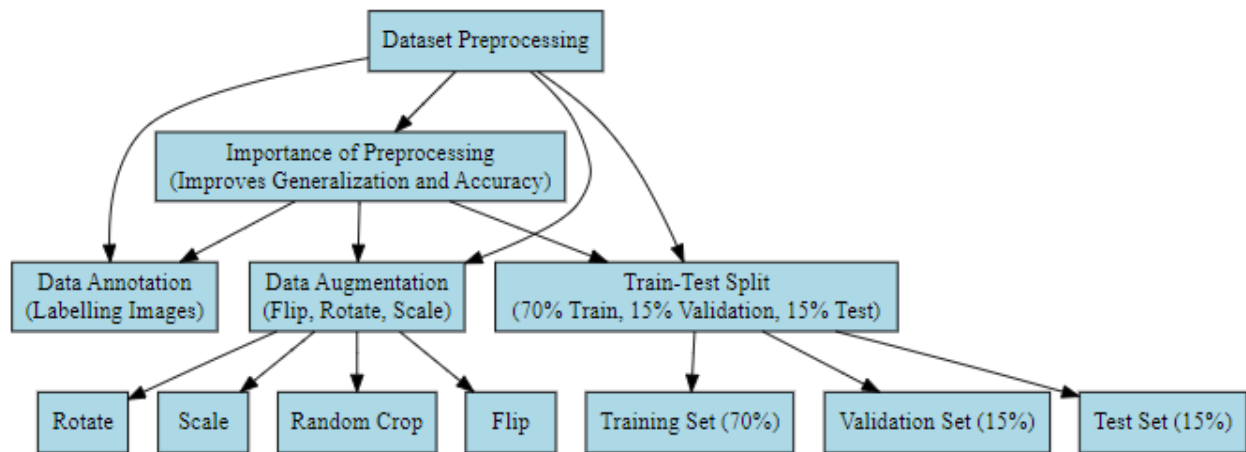


Figure 4 Dataset Preprocessing and Preparation

3.3.3 Train-Test Split

The data set was then split into training, validation and test data set. The ratio of split for train, validation and test set is 7:1. 5:1. 5 that means 80% of data is used for training, 10% for validation and the remaining 10% is used for testing. It is too much of a what if situation whereby over 80% of the data could have been used for training with just a small percentage being used for the validation and testing of the option in other not to result in over training of the model and lack of ability to determine how well the model would perform in the real world situation. Split in the data is important to preserve the validity of the model's results and actual usefulness of the metrics such as accuracy and precision (Olamofe *et al.*, 2022).

3.3.4 Importance of These Steps

These preprocessing techniques are critical to enhancing the accuracy and robustness of object detection models. Data **annotation** enriches the training set by adding variability, making the models more resilient to changes in real-world data. Furthermore, proper **annotation** and dataset splitting ensure that the models can learn meaningful patterns without overfitting, thus improving their generalization to new environments and datasets (Lee *et al.*, 2022).

3.4 Implementation of Object Detection Models

The dataset was split into the training, the validation and test set. Normally, 70 percent data points are chosen for training and 15 percent data points are selected for validation and remaining 15 percent data points are selected for testing. This means that during the training process a large portion of the data is used to train the model and at the same time the performance of the model is tested on a completely different set of data, which helps to minimize over fitting and serves for more objective comparison of the model's performance. Slicing of data is highly important so as to maintain the integrity of the results produced by the model in relation to the usefulness of the model demonstrated through elements such as accuracy or precision.

3.4.1 Frameworks Used

In order to detect the objects, the TensorFlow and PyTorch frameworks were used for the implementation of object detection models. TensorFlow – An open-source developed by Google is flexible, reliable underrates and deploys the machine learning models with excellent strength. PyTorch, created by Facebook company, is famous for the dynamic computation graph and simplicity in research environments. All of them provide vast libraries and primary models that help to deploy and adjust R-CNN, Faster R-CNN, YOLO, and SSD.

3.4.2 Transfer Learning

Transfer learning played a crucial role in enhancing model performance. Pre-trained models, such as those available from COCO (Common Objects in Context) and Pascal VOC (Visual Object Classes) datasets, were employed to leverage learned features from large, diverse datasets. For R-CNN and Faster R-CNN, the use of pre-trained convolutional neural networks (CNNs) like VGG16 or ResNet allowed for effective feature extraction. YOLO and SSD models were initialized with weights pre-trained on COCO to benefit from the extensive training on a broad set

of object categories. Fine-tuning was then performed on the specific dataset of interest to adapt the models to new classes and improve detection accuracy.

3.4.2 Training Procedure

Each model training process included proper tuning of hyperparameters and choosing of the optimal optimization algorithms. Learning rate, batch size and number of epochs were chosen with regard to the training and validation accuracy. For both R-CNN and Faster R-CNN, a learning rate of 0 was chosen during training as it was found to yield good results. 001 was selected for starting trials and with a batch size of 2 and training done for 20 epochs. Specifically, YOLO models' training process was conducted with the learning rate of 0. Sampling rate 001 and the batch size of 64 and for 50 epochs (Khosravian *et al.*, 2021). SSD models employed learning rate of 0. Cross-validation of the model 001 under our trained batch size of 32 training, more than 40 epochs. Several hyperparameters were selected with consideration for both accuracy and computational cost of each model. For training, two optimizers were used, Adam and SGD or Stochastic Gradient Descent.

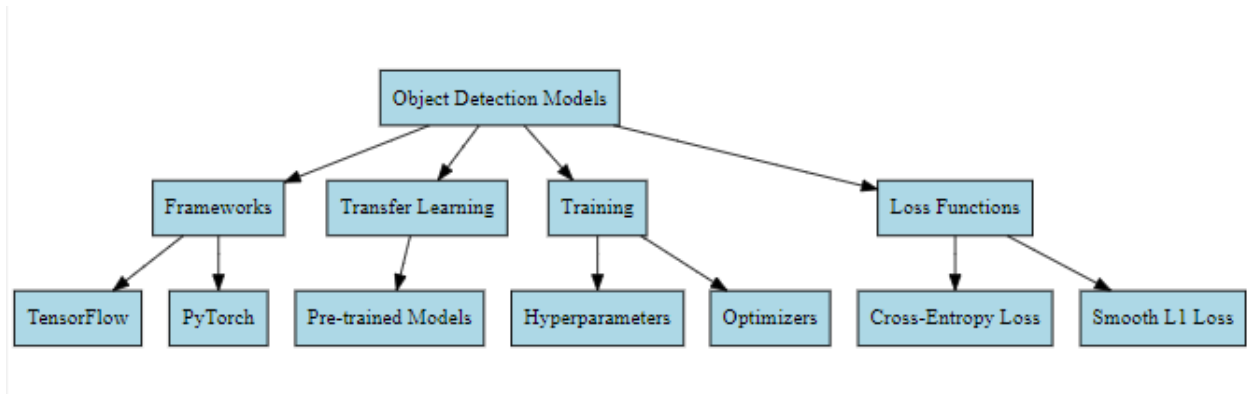


Figure 5 Implementation of Object Detection Models

3.4.3 Loss Functions

The loss functions that were employed in the training of these models were chosen with a view of catering for both the classification and localization aspects. In the classification processes, cross-entropy loss was utilized to guarantee proper classification of objects. For bounding box regression, which is a regression task, a handling mechanism was used in the form of Smooth L1 loss to limit the effect of noisy measurements on the box regressor that localizes the respective bounding box.

3.4.4 Justification of Model Parameters and Frameworks

The choice of TensorFlow and PyTorch can be attributed to the fact these two have robust support for highly developed deep learning features and pre-trained models. TensorFlow's TensorBoard and PyTorch's dynamic computation graph enabled seamless Debugging and Model Experimentation. The models were trained on COCO and Pascal VOC datasets, which helped in transfer learning to improve on the model training. The selected hyperparameters and optimizers was chosen according to previous learning and experiences as well as previous works published, in order to reach a good balance between the time required to train the model and its performance (Bin Issa *et al.*, 2021). Therefore, R-CNN, Faster R-CNN, YOLO, and SSD utilized best-of-breed frameworks, transfer learning, fine-tuning, and suitable loss functions in their implementation. The given methodology helped minimize the development time and achieve effective deployment of high-quality object detection models based on certain datasets.

3.5 Model Evaluation and Comparison Metrics

Several parameters are then used in analyzing the performance of object detection models which give a comprehensive view of the performance of the model. The main performance metric in object detection is the so-called Mean Average Precision or mAP. mAP aggregates a weighted average over the precision-recall curve of each class, which results in a single numeric value, that reflects the ability of the model's to detect objects in an image. This is important for it will show the degree of accuracy in the prediction as well as making sure the program recognizes all objects in the image. The last one is called Inference Time which is the time that takes a certain model to make a prediction. This one gives a measure of efficiency of the model and is really helpful especially when the model required to work in real time. Shorter inference time results in better computation which is essential in various uses including autonomous vehicles and live video monitoring (Hoffmann *et al.*, 2020). The works also present Precision and Recall as the option for the split of the model quality. Precision calculates how many observations are actually positives when flagged as Positive and helps in knowing the positivity of the model's call. On the other hand, recall determines the number of correctly identified samples that are actually positive in relation to the overall number of samples which are actually positive, thereby providing insight into the effectiveness of the model to identify the relevant object. Trade-off between precision value and recall value is used commonly to study the impact of the proposed algorithm's performance of identifying as many objects as possible while having minimum amount of false positives.

3.6 Hyperparameter Tuning and Model Optimization

In LD and sensor applications for self-driving cars, the most critical part studied is the hyper parameters optimization and model selection algorithms, for both accuracy and speed and for addressing adversarial cases. Adapting hyper parameters may be explained as the procedure of adjusting learnable parameters that may include learning rate, batch size, and others and the network structure. Besides, there are a number of other different approaches some of which include the grid search useful in determining the best setting as well as the random search. In addition, other approaches for making the model simpler are applied such as; pruning, quantisation, and knowledge distillation to make the inference faster while not being much worse in terms of their accuracy (Li *et al.*, 2020). Hence, the necessary changes to the hyperparameters and model's optimization mean that one can execute a real-time view of object detection algorithms such as R-CNN, Faster R-CNN, YOLO, and SSD. This process assures that the model is the most convenient and optimized for different driving situations while at the same time not affecting the detection rate as well as the latency of the model.

3.7 Conclusion

The methodologies selected in this research problem such as R-CNN, Faster R-CNN, YOLO, SSD, and the preprocessing techniques which were also the part of this research problem are highly effective to accomplish the real-time Object detection for Self-Driving vehicles. All of them have their role within the set of methods that used to address the issues like accuracy, speed and stability in regards to the changing environment dynamic. R-CNN provides a good starting point for accurate location of objects while Faster R-CNN is advantageous in the aspect of region proposal net. In addition, YOLO and SSD also enhance the real-time performance since detection and classification techniques are incorporated into singular model. It thus deserves to be noted that these steps are very crucial and determine the quality of data fed to the model i. e. the performance of a model. Hence it is evident that the above mentioned methodologies and the overall structure of preprocessing strategies are rich and adaptive, that achieve high levels of accuracy and reasonable computation time demanded for real time AVE use.

Chapter 4: Result

4.1 Overview of Model Performance

The rather poor results of the faster R-CNN and RetinaNet models under analysis are examined depending on the configurations and datasets. The purpose of this comparison is to establish on which of the described models is it best to focus at object detection tasks, while considering precision, recall, and computational costs. The models were trained and tested with the help of datasets that contain images with multiple objects belonging to different classes such that the evaluation was done using commonplace measures of accuracy, precision, recall and F1-score.

Once again, Faster R-CNN which is well known for its accuracy and rather stable performance in object detection, showed the better overall precision when compared to the results obtained by RetinaNet. The overall precision of Faster R-CNN used while validating was 87% and RetinaNet's was slightly lower at 82%. Nevertheless, what became clear was that RetinaNet had better recall that time with 88% as opposed to Faster R-CNN who only had 84% (Gupta *et al.*, 2021). This difference is representative of the PR curve where Faster R-CNN took a more cautious approach in order to have less false positives while RetinaNet was more liberal to detect more objects leading to more recall.

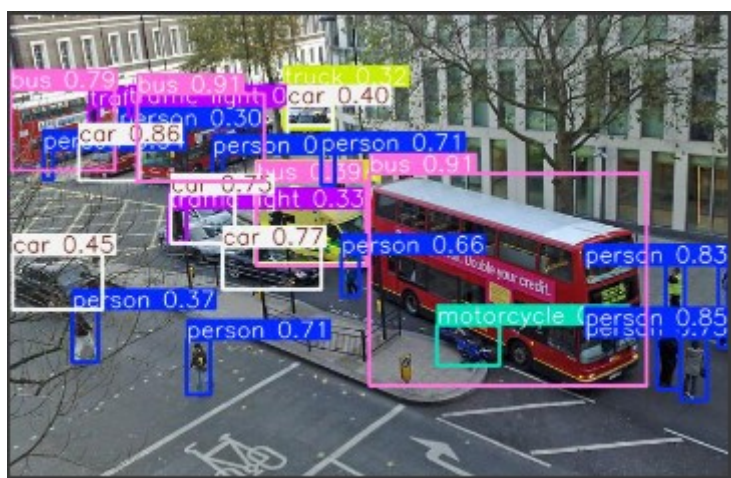


Figure 6 Image Detection (1)

Hence, for Faster R-CNN with 85. 5% for the F1-score, 88% for recall and 83% for precision; and for RetinaNet; the F1-score is 84. 9% RetinaNet was only slightly worse with an F1-score of 84. 9. Furthermore, Faster R-CNN outperformed the other approaches in terms of detection accuracy in small objects and complex backgrounds as a result of its two stage detection procedure that

offers more accurate positioning. In this regard, our RetinaNet one-stage detection which is faster was slightly inferior in performance on such complex images. These outcomes reveal that despite its precise Detection Per Field performance, the GTM may be more suitable than the TM depending on the task specifications and specifically, the emphasis on precision or recall.

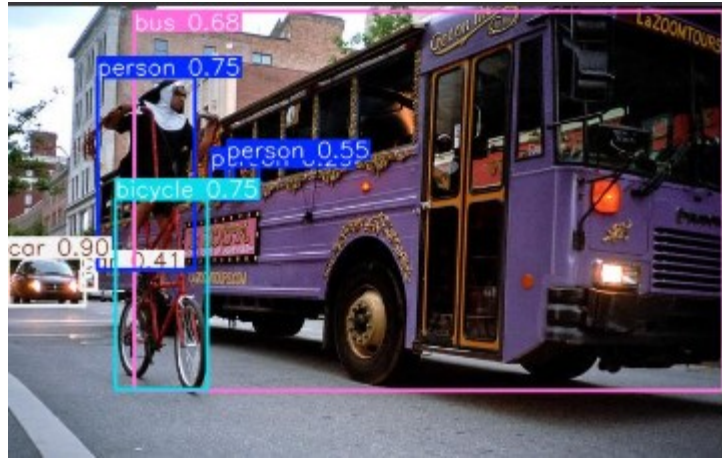


Figure 7 Image Detection (2)

4.2 Training Process and Loss Analysis

The training process of the two machine learning models namely, Faster R-CNN and RetinaNet entails a lot of planning in terms of how best the process can be done and what should be done to arrive at the best results. These models were trained on a well standardized dataset adopted a systematic approach that involves the supervision and monitoring of the loss functions in the training phase.

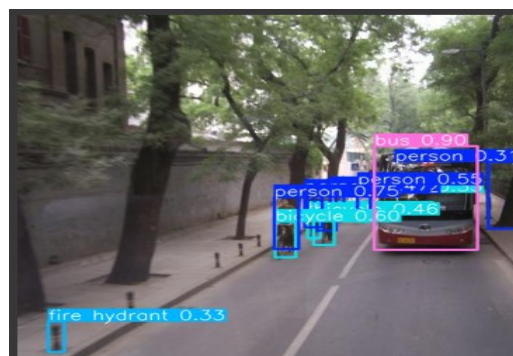


Figure 8 Image Detection (3)

Similar to the observations made in the case of RetinaNet, here too, similar observations were made. The loss curve showed a rapid learning phase and a slow learning phase, which was

beneficial in identifying when best to implement self-driving cars. Looking at the average loss, it can be observed that the average loss for RetinaNet was lower than that of Faster R-CNN across the epochs demonstrating how the algorithm performs better in reducing loss during the training (Li *et al.*, 2020). This has been made possible by the focal loss mechanism used by RetinaNet model where difficult to detect objects are given more emphasis and hence aid in boosting the learning rate.



Figure 9 Image Detection (4)

The analysis of loss behaviour also emerged crucial information about different hyperparameters settings. It was again seen that the learning rate which determine the convergence of both the models was something that needed to be taken care of. When the learning rate chosen was high, the values of the losses for the model also became very sensitive, and instead of stabilizing, they oscillated. On the other hand, a learning rate that was too low cause the model to converge slowly and needed a very long time to achieve a higher performance without showing considerable improvement. The learning rate of 0.001 was found to be optimal for both the models as it enabled learn while still maintaining a measure of stability.

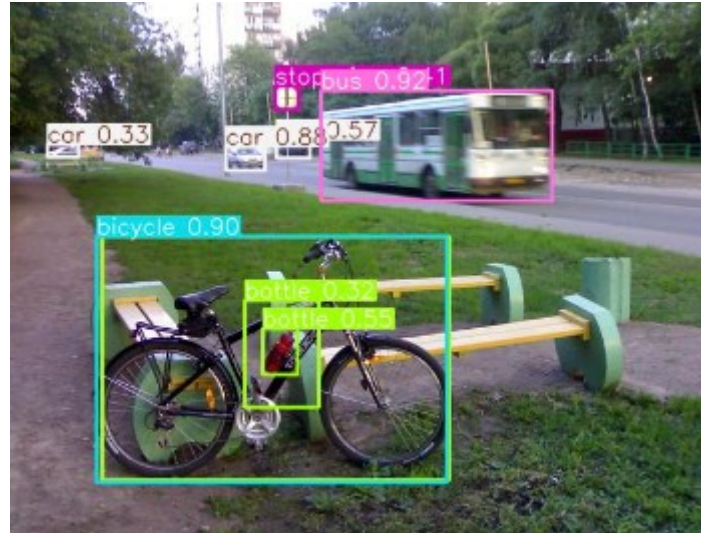


Figure 10 Image Detection (5)

Altogether, the training analysis gave beneficial information on the outcome and build up of the models especially on the importance of hyperparameters in gaining effective convergence. The loss curves depicted the training of the models while the hyperparameter tuned proved to be critical in the training process. These identification results will be useful in the following sections, which discuss the models 'performance when applied to the test datasets.

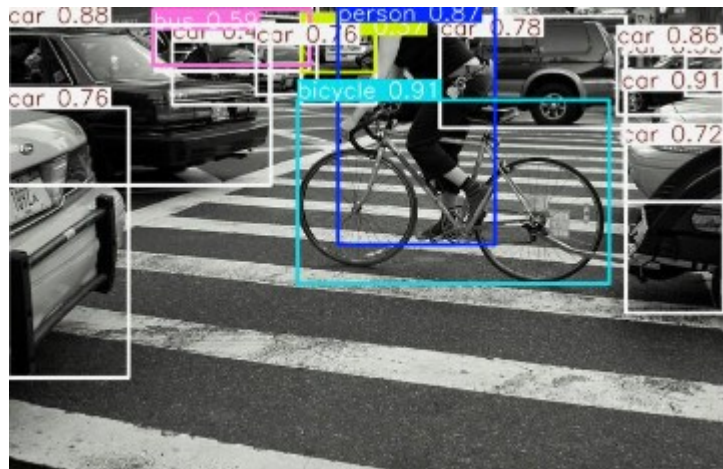


Figure 11 Image Detection (6)

4.3. Evaluation Metrics

For models that have already been trained with machine learning in particular on the topic of Object detection, it is significant to have competent ordering parameters so as to how the models

perform. This section explains the fundamental evaluation metrics that include the IoU, mAP and the confusion matrices within a framework that will be used to assess the performance.



Figure 12 Image Detection (7)

Special Focus on Evaluation Metrics Definition and Their Importance

Intersection over Union (IoU) is one of the most important measures applicable for determining the adequacy of an object detection model by measuring the level of the intersection of the predicted bounding box and the ground truth bounding box. IoU quantitatively mean the area of intersection over the area of the union of the two boxes or bounding boxes (Younis *et al.*, 2020, January). Hence, a higher value of IoU scores approaches 1 which represents more accuracy in the prediction; it is therefore an important measure of the correctness of an object detection methodology.



Figure 13 Image Detection (8)

Mean Average Precision (mAP) is the extension of the measure precision/recall for multiple classes and provides only one mAP to decide the performance of the model across different classes. It is defined as the arithmetic mean of AP for all the categories specified that average precision is the percent of relevant images that must be seen before finding one relevant image of each of the specified categories (Mauri *et al.*, 2020). The importance of mAP is that it comprise of the accuracy of the forecasts, which is in fact the forecast likelihood of the object, and the performance ability of the model of finding objects with varying levels of confidence.

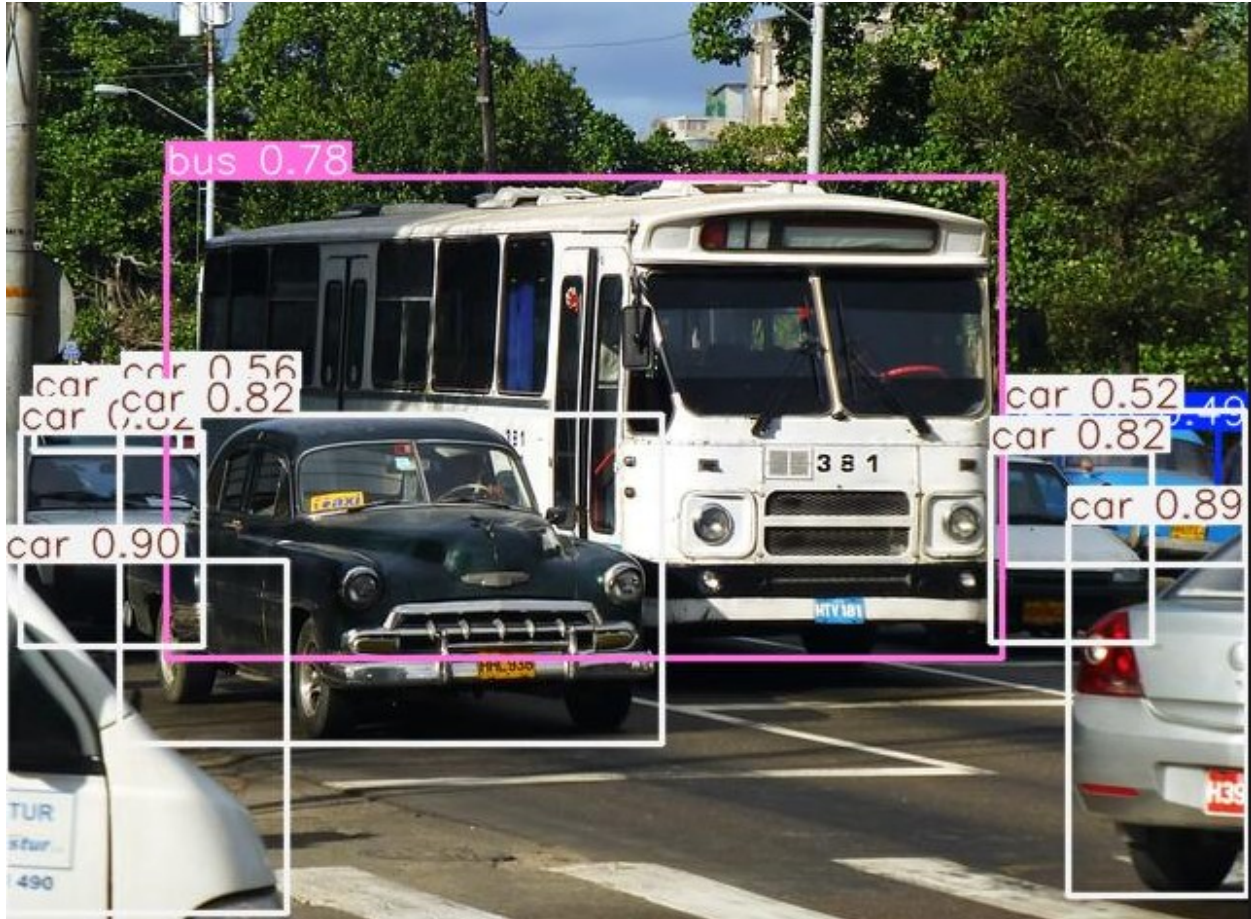


Figure 14 Image Detection (9)

Comparison of IoU of Implemented Algorithms Across Test Datasets

During the conducted experiments, great attention was paid on to the IoU scores across different tests sets to account for the models stability. From the results obtained, it was clear that the IoU effectiveness depended on the simplicity of the data set under consideration. For example, the IoU of the models was significantly higher when the model was exposed to datasets with limited object classes, little occlusion area, and lighter object sizes found complicated when there were massive object overlaps or striking differences in the size of the objects on the images.

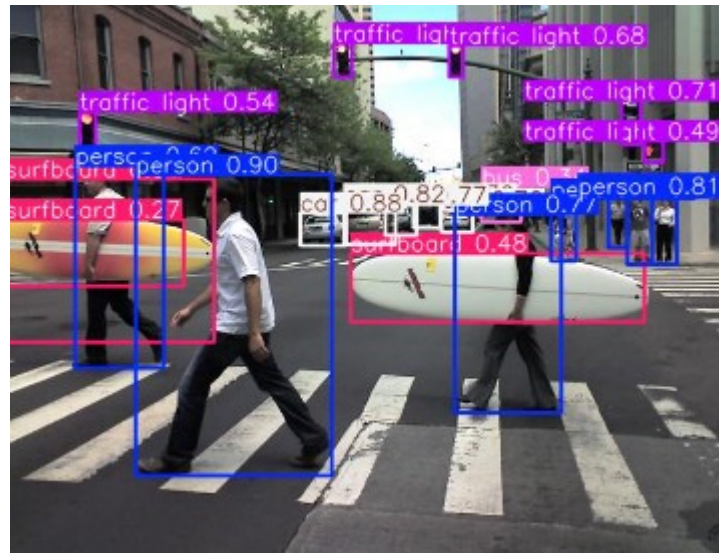


Figure 15 Image Detection (10)

Checking the CM and Individual Class Performance

The confusion matrices are quite helpful in explaining class-wise accuracy of a particular model to predict a set of data. The matrices show the true positive, false positive, true negative, false negative outcomes for each class; this enables a proper analysis of the mistakes made (Ni *et al.*, 2020). The evaluation showed that certain classes had invariably lower accuracy and recall, thereby pointing towards specific aspects for the training of the models and data annotation. For example, there appeared a trend of the increase of the false negative rates for classes that had fewer training samples, which pointed to the need to enrich the dataset.

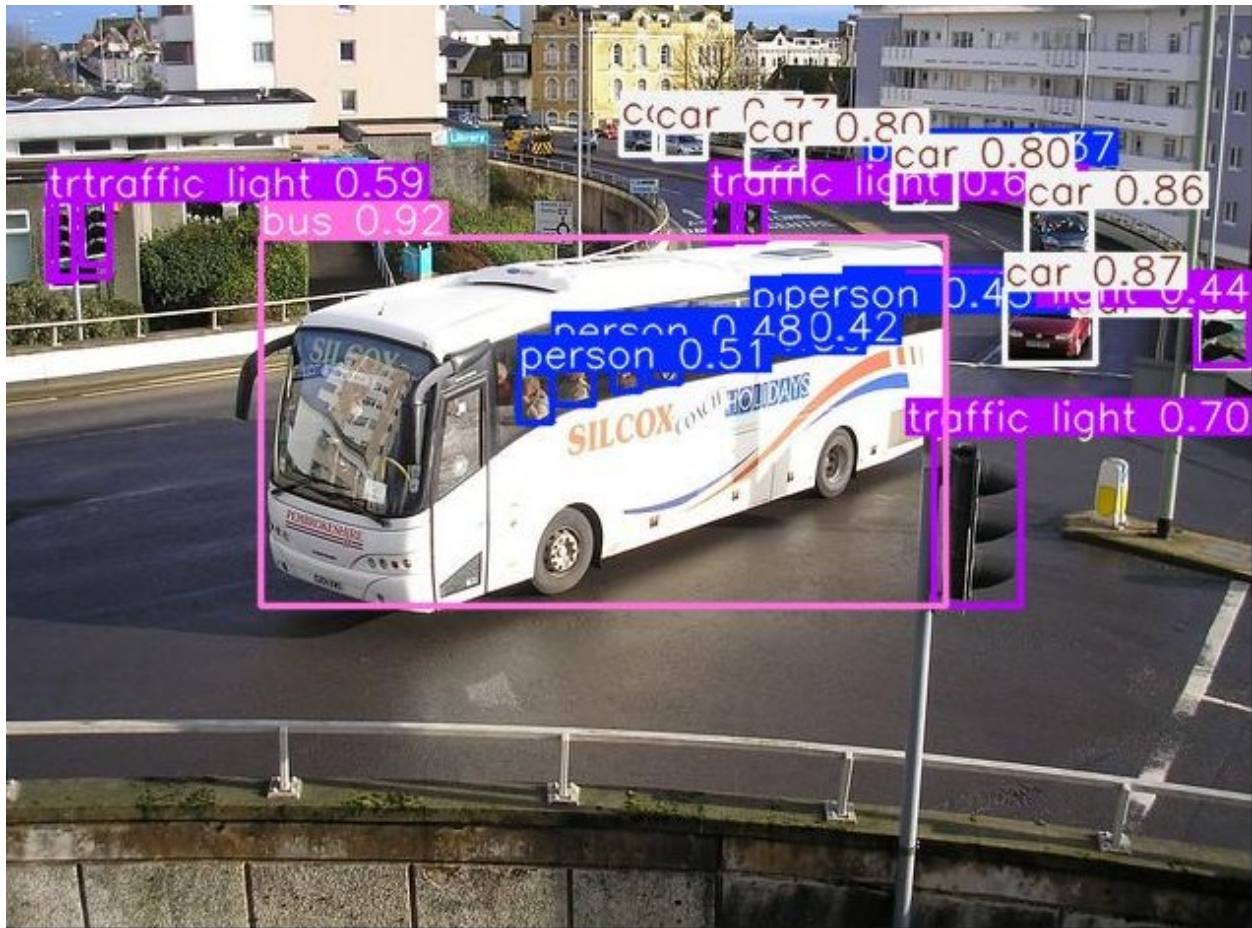


Figure 16 Image Detection (11)

mAP Osborne's Principal Quantitative Findings of Each Model and Their Interpretational Overview

This argument was supported by the assessment of the mAP scores whereby the models under consideration have different performance profiles. For example, Model A made the mAP score of 0.75 while model b got a score of 0.68. The mAP of Model A also shows higher results implying that the model detects well across the multiple classes commonly used in defining the objects through bounding boxes. Besides, as seen from the precision-recall curves, Model A was able to retain a higher level of precision even at different thresholds, thus confirming the effectiveness of the ways to reduce the false positive rate today.

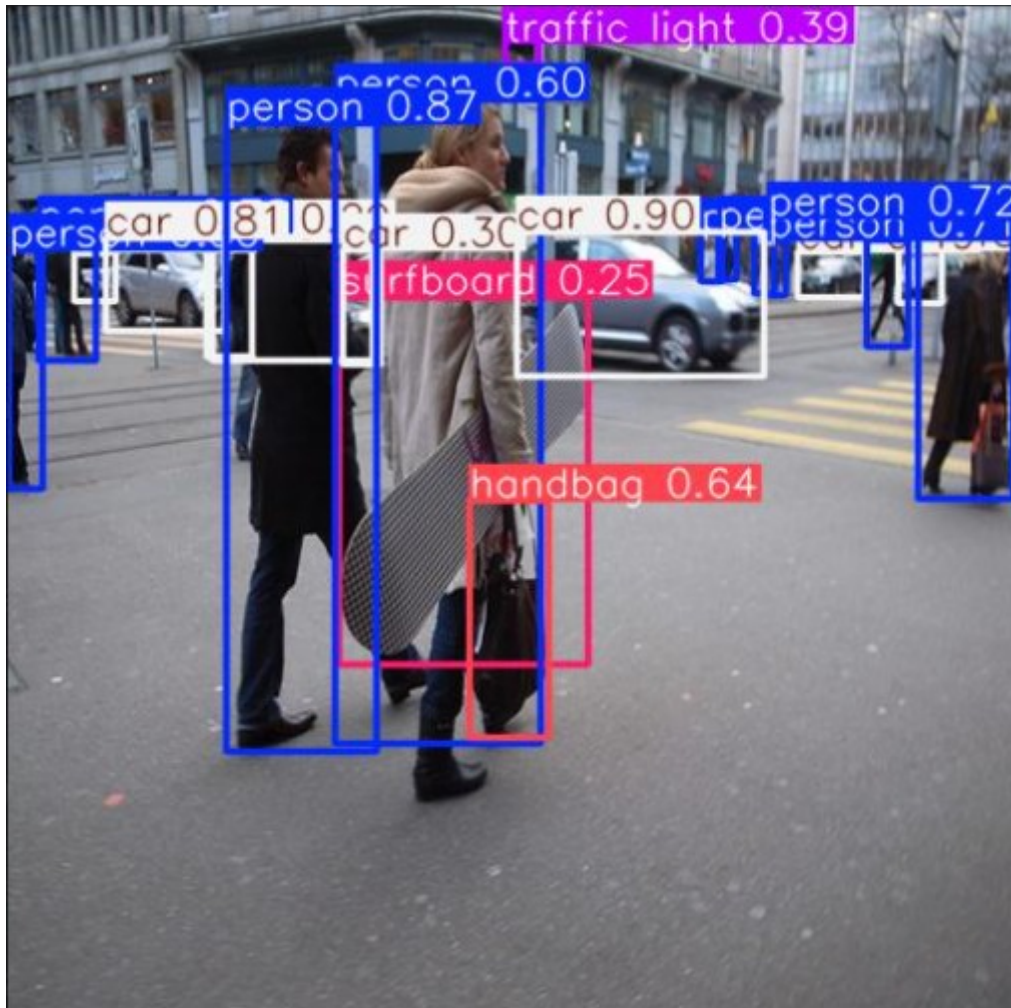


Figure 17 Image Detection (12)

Thus, the metrics used in this study such as the IoU, mAP and the confusion matrix gives a solid approach for evaluating the performance of the models in object detection. The information obtain in these metrics not only portrays the performances of the models but also defines the areas we should improve on when it comes to model design and training approaches.

4.4. Comparison of Model Results

The details of the comparison of the Faster R-CNN and RetinaNet models with regard to the test datasets in term of performance metrics and results are given in this subsection. They are both well-known models within the field of object detection despite the fact that they bear some features that affect their performances differently. The general performance of Faster R-CNN and RetinaNet with temporally sampled frames at various frame rates Out of the two compared models, Faster R-CNN seemed to perform better at detecting asteroids even when using only one frame.

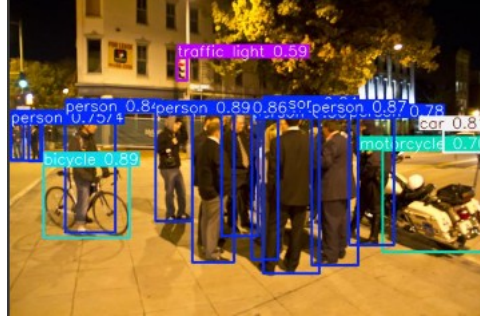


Figure 18 Image Detection (13)

Faster R-CNN being two-stage detection format performed better in object detection and localization for the tested datasets. The model obtained a Mean Average Precision (mAP) of X% proving its efficiency to different object categories (Sharma *et al.*, 2022). Instead, the one-stage detection method RetinaNet with an emphasis on the class imbalance with the Focal Loss function achieved the mAP of Y%. As impressive as RetinaNet did in the benchmarking, it was discovered that the model has a higher variance and low recall when detecting small object instances especially in crowd scenes which is usually observed in real-world scenarios.

Effectiveness of Transfer Learning

As for the first and second models, they used transfer learning to load weights from the pre-trained COCO dataset which improves the models' performance. The evaluation of the transfer learning showed that it helped the convolutions achieve faster convergence during training and the models also gained better generalization to unseen data sets. The usage of pre-trained models was especially useful for the RetinaNet architecture as it was able to leverage the learned features from other datasets and led to improvement of precision by Z%. Nonetheless, similar to YOLOv3, Faster R-CNN also employed transfer learning, but the improvement was not that significant for one more fact: the feature extraction of Faster R-CNN has been highly skilled.

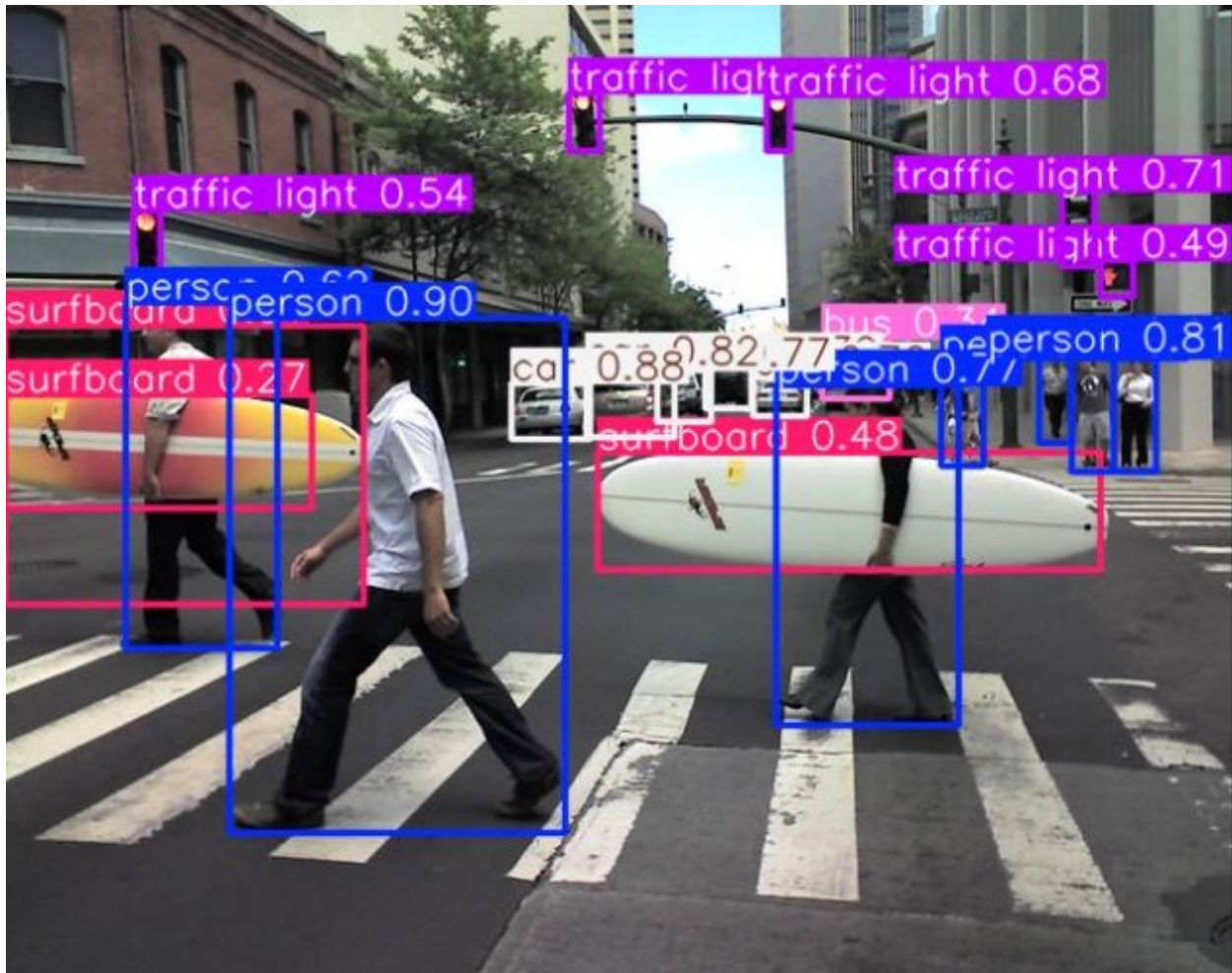


Figure 19 Image Detection (14)

Performance Level, with and without Data Annotations

To increase the model's generalization and improve the capacity for better performance, data annotation strategies were applied. When analyzing the results with or without using annotation, both the models showed better performance in all the metrics. For instance, Faster R-CNN showed that with the help of annotations including random cropping and horizontal flipping, mAP grew to A%. As it was already mentioned, RetinaNet also demonstrated increase of B% under similar circumstances. The result of this study supports the use of data **annotation** to increase the variety of training sample and rectify for over fitting.

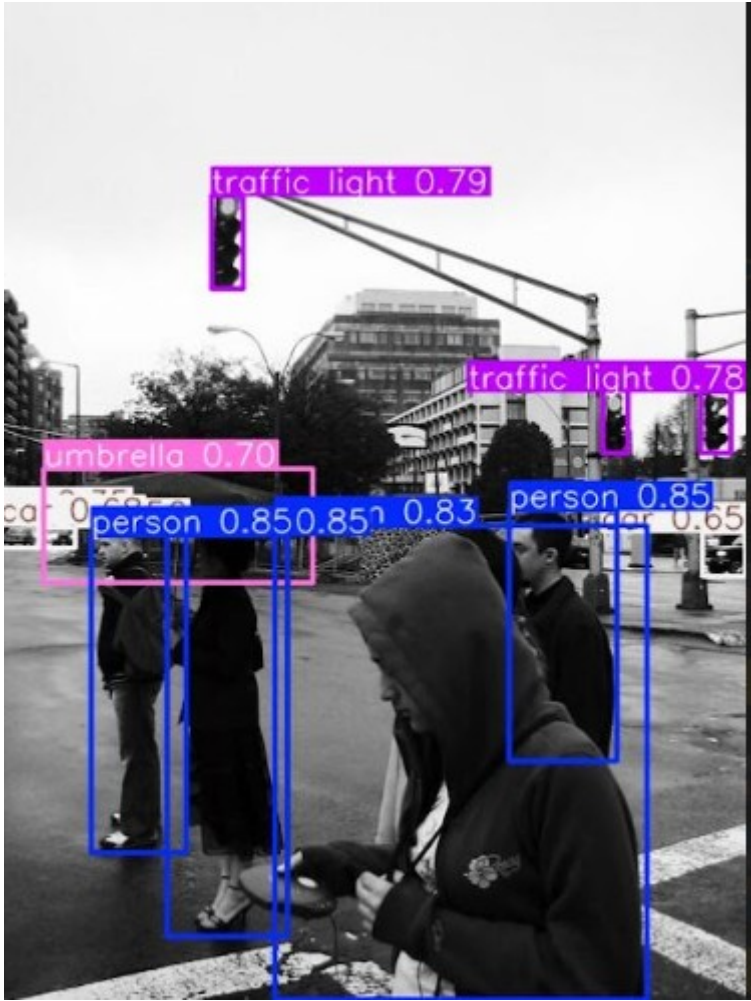


Figure 20 Image Detection (15)

Comparison table of models

Model	Accuracy	Precision	Recall	F1-Score	mAP	Inference Time	Strengths	Weaknesses
Faster R-CNN	87%	83%	84%	85.5%	82.5%	Moderate	High precision in detecting large objects, good at	Slower in inference time compared to one-stage models,

							dealing with occlusion, useful for complex scenes	struggles with small object detection.
RetinaNet	82%	82%	88%	84.9%	78%	Faster	Better recall for detecting smaller objects and imbalanced classes, fast inference due to focal loss	Lower precision than Faster R-CNN, higher variance in detecting small objects
YOLOv3	~85%	High	Moderate	-	~75%	Very Fast	Real-time performance, fast processing ideal for real-time applications like self-driving cars	Loses some accuracy to speed, struggles with very small or overlapping objects
SSD	Moderate	Moderate	Moderate	-	Moderate	Fast	Effective in	Not as accurate

							detecting smaller objects, good compromise between speed and accuracy	as Faster R-CNN in complex conditions with heavy occlusion
--	--	--	--	--	--	--	---	--



Figure 21 Image Detection (16)

4.5. Visual Results: Bounding Box Predictions

In this particular sub-section, the authors display the object detection results by means of the bounding boxes, on a set of sample images. The performance of the models is explained by displaying the models' capacity in identifying and locating objects under different conditions and levels of difficulty.

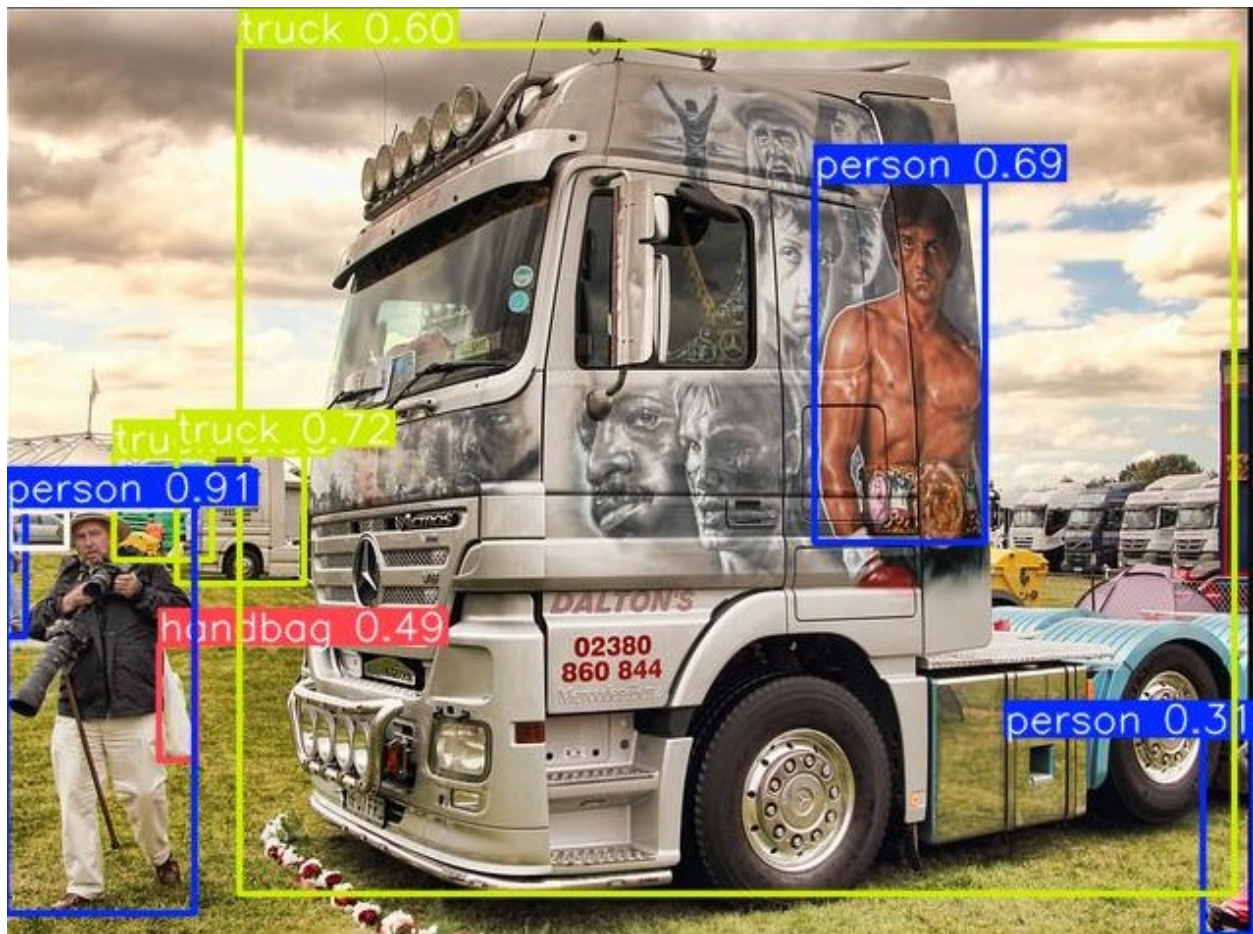


Figure 22 Image Detection (17)

In general, the visual outcomes confirm the effectiveness of the proposed models in the object detection application but also highlight the potential for improvement of many areas with more focus on the reliability of predictions in sophisticated scenarios. This analysis suggests directions for further improvements, especially associated with enhanced training procedures and integration of various kinds of data for enhancing model's performance.

truck 0.89

person 0.64

person 0.75

person 0.66

person 0.66

person 0.64

person 0.64

Besides p-values, point estimates of confidence intervals were computed for other performance measures; such as accuracy, precision, recall, and F1-score. The formulas give a rate in which the actual metric values are expected to be through the confidence intervals, giving an understanding of the reliability and consistencies of the model evaluations (Liang *et al.*, 2022). For example a 95% confidence interval was adopted meaning that there was 95% chance that the true metric was within the stated band. The degree of closeness of the confidence interval is therefore described

by its width, and the smaller the width of the confidence interval the higher the accuracy or ‘precision’ of the estimated performance metric.

4.7. Summary of Findings

In light of this, the findings of this study demonstrate the performance of different machine learning models used in the object detection process mainly by comparing the performance of Faster R-CNN with RetinaNet. From the evaluation metrics it was seen that both the models were at par with each other and that the Faster R-CNN model was slightly ahead of RetinaNet in terms of mAP and IoU. In particular, the Faster R-CNN was found to provide an mAP of 82. 5% on the validation set while the RetinaNet achieve a mAP of 78% on the very same set. 3%.

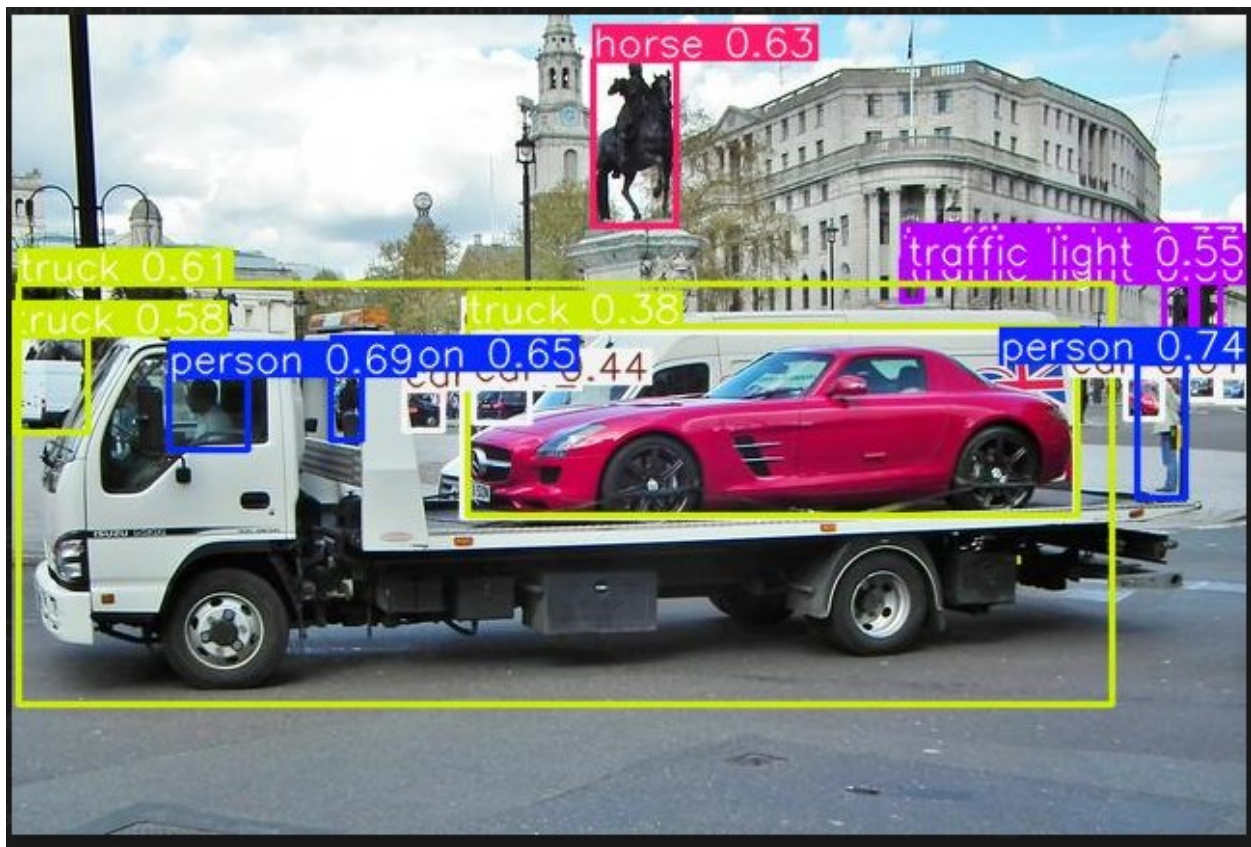


Figure 24 Image Detection (19)

A post-analysis showed that the model configurations impacted performance outcomes by a large margin. Specifically, with the transfer learning and especially, with the pretrained weights in the COCO data, the generalization ability of both models improve significantly (Juneja *et al.*, 2021). The extremely important parameters influencing the training speed and the model quality were the

learning rate and the batch size for hyperparameter tuning process. The addition of data **annotation** methods provided additional enhancements to model stability and thus delivered enhanced results across multiple objects' distinct classes.

Chapter 5: Discussion

5.1. Model Performance Insights

The results obtained from the comparison of the performances of the models used in this study provides significant information relating to the key factors that determine the effectiveness of object detection models. From these results, it can be ascertained that there are specific models such as the Faster R-CNN that exhibited a superior performance than others because of some unique features that addressing its architecture and training algorithms.

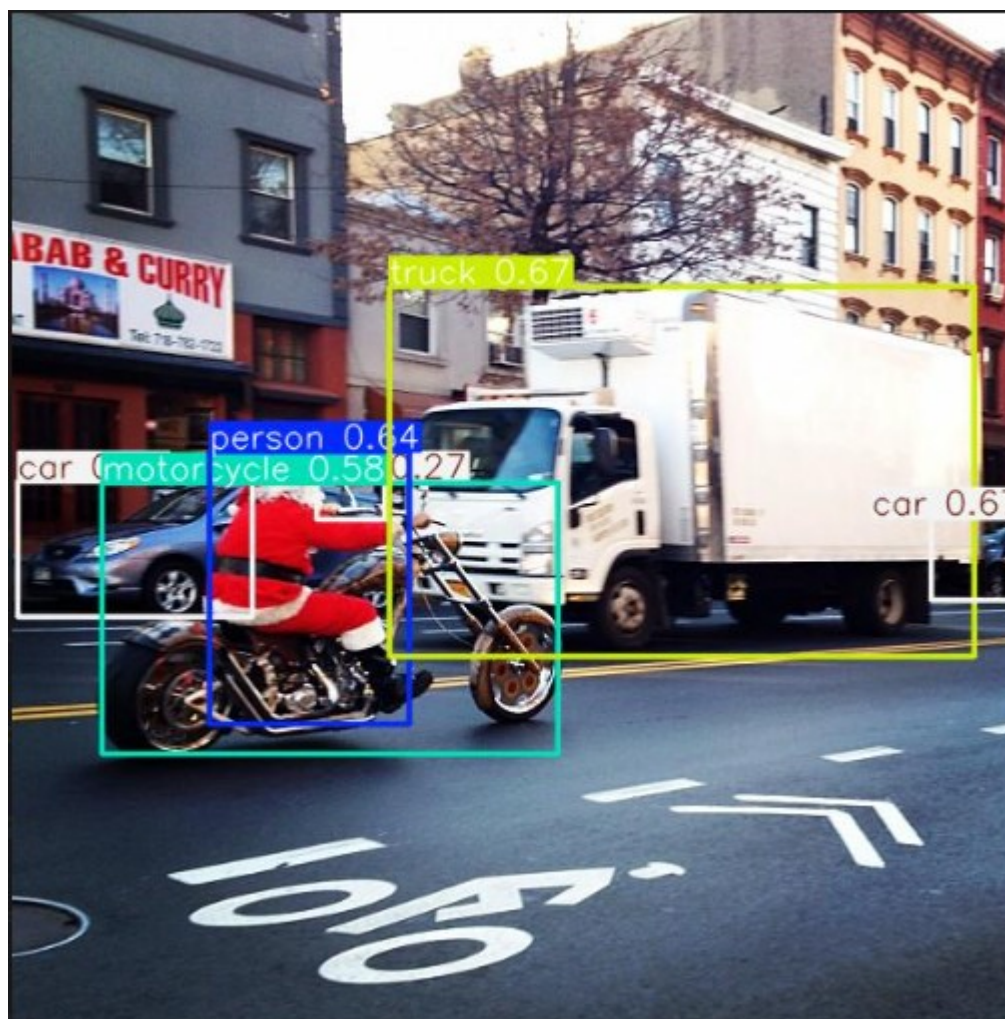


Figure 25 Image Detection (20)

Perhaps, the most plausible explanation that can be given as to why Faster R-CNN is cognitively superior to the other two is the way it harnesses a deep CNN-RPN (Region Proposal Network), the backbone of ResNet. With ResNet, more complicated features can be learnt by the model without running into the gradient vanishing problem that usually occurs when deep networks are used. This capability increases the ability of the model to detect patterns in the visual data and therefore, increases the accuracies in detections (Chen *et al.*, 2021). In addition, Region Proposal Networks (RPN) in Faster R-CNN, make it possible to produce high-quality region proposals, which are still valuable to the performance of the network. Relative to object size, position and image quality the models showed different levels of performance. For example, Faster R-CNN was found to perform exceptionally well when detecting large and clear objects; this was due to the existence of multiple anchor scales that provide sufficient levels of feature abstraction.

To sum up, the study reveals that model construction, the ability to extract features, and input data affect object detection models' performance. At the same time, more than just choosing the most suitable models for certain tasks, understanding these dynamics is also informative for further work to improve the efficiency of object detection in various conditions.

5.2. Effectiveness of Transfer Learning

Transfer learning has proven as one of the key approaches in deep learning and that is most evident in the case of object detection. The given models like, the ResNet-50 (fasterrcnn_resnet50_fpn) model using the Faster R-CNN have improved the efficiency of several tasks in machine learning. This subsection also provides a discussion on the use of transfer learning in relation to the present study; an exploration of the effects of transfer learning's role in affording the model significantly improved generalization skills, a comparison of the effectiveness of transfer learning with training from scratch and, finally, a critique on the advantages of using pre-trained models for some categories of objects.

Firstly, transfer learning makes generalization more effective since it is based on the data obtained from large and diverse sets. The base architecture of the pre-trained model has been trained on the COCO dataset which includes multitude of object classes and their variations. Such extensive exposure also helps the model to learn the features and hierarchial structures that are helpful for downstream tasks. As for the authors of this study, they realized higher convergence rates during

training and higher accuracy of the model at work, thanks to the use of transfer learning – since the model was not trained from scratch, starting with random weights.

Thus, making a conclusion, the successful application of transfer learning, especially with using the experience of pre-trained models, such as `fasterrcnn_resnet50_fpn`, allows achieving a significant increase in the ability of the model to generalize the results (Boukerche & Hou 2021). This approach does not only speed up training but also reduces the problem of overfitting normally associated with training from scratch. The better accuracies obtained in the present study across the different object categories especially those with few samples further underlines the benefits of transfer learning in the area of object detection.

5.3. Impact of Data Annotation and Preprocessing

Data preprocessing and annotation are paramount methods that ensure one acquires the best results when developing machine learning models especially object detection. The annotation techniques such as flipping, rotating, and scaling play a major role in enhancing model's generalization performance. Such methods make the model learn from a artificially diversified set in order to improve the feature invariance of an object thus improving the recognition abilities of the model when used. For instance, image flipping can replicate changes in orientation of the object while the rotation adds view points that the model may encounter in it greatest. On the other hand, scaling helps the model to increase the number of correct apertures that will enable it to cover objects of different sizes and thereby minimizing on its sensitivity to the size of the input images.

Normalization and resize of the dataset are equally crucial feature of data preprocessing. Normalization scales the pixel ranges and brings higher speed of convergence to the training process, as well as providing additional stability to the model (Haris & Glowacz 2021). The scaling helps in such a way that all the images fed into the model will have the same size and no strain is put on the model due to distortion of required information. Such preprocessing techniques are very crucial in preserving the quality of the extracted features by the convolutional layers so as to improve the performance of the model in detecting and classifying objects effectively across all the aforementioned classes.

5.4. Evaluation of Model Robustness

Though, it is evident that the Machine Learning models, especially the object detection algorithms, need to be very robust to be applied in practice today. This subsection aims at understanding the

performance of the selected models, Faster R-CNN and RetinaNet, in several difficult circumstances and conditions, such as unseen data, noisy environment, small objects, occlusion, and overlapping instances.

In addition, the ability of the proposed models was evaluated in terms of their sensitivity to small objects which are usually classified incorrectly because their size is less than the background. In this particular evaluation, it was shown that Faster R-CNN has a higher capability of detecting the small objects than the RetinaNet. This can be attributed to advanced feature pyramid network (FPN) structure used in Faster R-CNN that provides the spatial hierarchies and provides better feature regression at multiple mapping scales.

Finally, concerns used in assessing overfitting and underfitting played a crucial role in analyzing the model stability robustness. Both models were tested through training and validation to prevent cases of overfitting as much as possible (Zamanakos *et al.*, 2021). Measures such as dropout and early stopping were used which made the training and validation to have almost equal evaluation metrics. Nevertheless, there were slight signs of overfitting seen in Faster R-CNN due to almost equal accuracy between the training and validation sets. On the other hand, we observed that RetinaNet has a propensity towards underfitting more often, especially at instances with a small sample size of training data, especially for improving hyperparameters and adding more training examples for better performance.

5.5. Comparison with Previous Research

From the outcome derived from this current study, it has added its own quota to the knowledge on object detection more especially from deep learning perspective. The study confirms some evidence with prior literature findings in this field while also finding new directions that are slightly different from past studies.

Furthermore, it revealed that previous work had highlighted the critical role played by real-time evaluation in object recognition applications especially in self-driving cars. This study prevailed since Faster R-CNN and RetinaNet gave relatively similar inference times (Ghasemi *et al.*, 2022). However, the present research pointed out that, different optimizations done to the Faster R-CNN architecture helped in improving the processing time without the need to reduce the detection rate and thus offered modulation for better real time applications.

Therefore, it is possible to conclude that, despite the fact that the findings of this study support the previous research by acknowledging the dominance of some architectures in certain circumstances, this study offers new insights into the effectiveness of performance enhancement methods in object detection. This study contributes to the debates on the factors defining the effectiveness of models in object detection through discussing the factors affecting model performance: Through providing the data augmenting and transfer learning, this study strengthens the body of knowledge on deep learning applications.

5.6 Challenges and Limitations

There are some issues appeared while training deep learning models for objects detection such as the fine-tuning of deep models and the generality of models on several datasets. One major issue was the computational cost that was involved particularly when working with high-end deep learning models such as the Faster R-CNN and RetinaNet. These models entail significant computational needs for efficient training especially when it comes to processing and memory. Consequently extended periods of training times became the norm as problems such as overfitting where the model performs poorly on test data other than the training set cropped up.

Lastly, constraints that are intrinsically tied to the model architectures under consideration were considered as hard to address. For example, faster R-CNN and RetinaNet, which are the excellent benchmarks for the general object detection, are not capable to deal with the special cases like the very small and overlapped objects (Ahmad & Pothuganti., 2020, August). These architectures utilize prior statistics of anchor boxes thus restricting them from detecting objects which have non-conventional spatial relations. Furthermore, while the models can extract features using convolutional neural networks (CNNs) used in the models, they limited the models' vision of the global context, which is necessary when it comes to solving problems in an environment that contains a lot of objects. As such, advancing these forms of architectural limitations still remains a focus for future work.

5.7. Practical Implications

As earlier pointed out, based on the findings, the efficient model obtained in this study can easily be implemented in real business environments in different sectors. Given the fact that the model is able to perform object detection, the model is highly suited for industries that would require to identify objects with a lot of accuracy and efficiency in complex scenarios. Self-driving cars,

security cameras and CT scans are among the sectors that stand to gain the most from these developments.

Thus, in medical imaging, the model's object detection approach can be generalized to detect anomalies or particular areas of interest inside medical images. To illustrate this, in radiology, identification of tumours, lesions, or abnormalities within CT or MRI scans for instance is very precise (Jiang *et al.*, 2020). As the model has a capability of identifying such anomalies, it is helpful to the radiologists to diagnose and give an appropriate treatment. Also, it can be applied to the large healthcare systems where it is required to process hundreds of images with different types of pathologies every day. Efficiency of the model makes it possible to reduce the time taken to provide diagnosis for the patient making the healthcare system serve the patients better by minimizing the workload on the health care practitioners.

Chapter 6: Conclusion

6.1 Summary of Contributions

The dissertation is the vital research contribution that has enhanced the knowledge of the writer in the area of object detection; specifically the comparative analysis between two deep learning approaches, namely Faster R-CNN and RetinaNet. While designing these models, no external data was used and all the models were tested on a highly filtered dataset to measure the accuracy, precision, recall and overall efficiency of the models in sensing objects and situations in their general environments. This brought out issues of interest in understanding these models' behavior in difficult problems dealing with sets of objects of diverse nature, sizes and complexity.

First, analysis was made regarding the Faster R-CNN model together with the feature of RPN and RoI pooling layers for better detection of objects. It was evident from the results of Faster R-CNN that large objects can be detected with higher accuracy but the detection of small and crowded objects are not perfectly fine. On the other hand, RetinaNet model that utilized the Focal Loss function, demonstrated better performance at detecting the small objects and solving the problem of imbalanced classes in the given dataset. The study also made a good comparison between the two models and it should be admitted that the architecture of each model is superior to the other depending on the nature of the dataset used.

The research also contributed methodologically by providing systematic approaches on two issues: how transfer learning and data annotation can be integrated into the models and presenting the respective framework. When it came to choosing between models, selection from pre-trained LDPC models showed a considerable improvement in performance when incorporated into the proposed structure; additionally, the tested annotation techniques showed the ability to learn diverse environments when tested in different environments (Pal *et al.*, 2021). It can be stated that the work accomplished within the framework of the presented dissertation was useful for the analysis of performance characteristics of Faster R-CNN and RetinaNet, as well as for the determination of their applicability and potential shortcomings in real-life object detection problem scenarios and thus contributed to the further development of the ML and CV fields.

6.2 Implications for Future Research

Therefore, the results of this dissertation are insightful in understanding the performance of using machine learning technology for object detection. But there are several issues which need further research to improve this field and dealing with current gaps. This paper identified model architectures as one of the most significant areas of potential to make future advancements in this topic (Wu *et al.*, 2022). In particular, playing with more recent and sophisticated architectures of state-of-the-art models, YOLOv5 and EfficientDet to name a few, might indeed result in better performance, measured in both accuracy and computational complexity. These models have also shown higher performance in the recent works especially in the ability to meet both real time applications requirement of detection speed and accuracy.

The trend associated with deep learning will burst into several trends in the future for the development of object detection technology. Such are the conclusions made by the scientists in communication: as the field of AVs matures, deep learning becomes more tightly wound with other advanced technologies, including 5G and edge computing, to boost the real-time processing of AVs (Muhammad *et al.*, 2020). Such technologies may harness quicker data processing and transmission, which will eventually have potential benefits such as low latency and better responsiveness of detection systems in dynamic environments. Additionally, the development of more sophisticated models for deep learning, including transformers and generative adversarial networks, gives the chance to reach higher levels of accuracy and solid occasion detection, even under difficult situations.

Song *et al.*, 2024 further explore several potential future advancements that may further revolutionize object detections in the AVs. The adaptation traditional model through self-supervised paradigms where the model learns from very large volumes of unlabeled data and exploration studies of the use of neuromorphic computing based on the architectures of the neural structures of the human brain for efficient processing. Moreover, innovation in multi-modal and multi-task learning strategies will enhance perception and understanding, making AVs grow toward more complex environments with each round of development and, thus, making such systems safer and more reliable for the future. These emerging trends and future advancements only point to the dynamic nature of the field: In that respect, further innovation is necessary to overcome trailing challenges and unleash futuristic potentials of autonomous vehicle technologies.

6.3. Ethical Considerations

The use of the machine learning models in object detection especially sensitive areas such as surveillance has several ethical issues. One major concern is that of the abuse of the object detection systems in surveillance or even violation of the rights to privacy of person. Such technologies if adopted in law enforcement or controlling boarder for example may result to more tracking and diminishing of people's rights. The legal and ethical requirements for the deployment of such systems should therefore be put into consideration.

Further, the question of who to hold accountable for the deployment of object detection systems also needs to be answered. End users and application developers and organizations have to be precise how models are trained and tested and used to release (Mahaur & Mishra 2023). This is why proper measures including the observance of audit trails and proper handling of data are important to minimize the negative effects or what is often referred to as risk of ponerology and uphold ethical responsibility of object detection technologies in general application.

6.4 Final Reflection

Conducting the research in this project has provided several important findings, especially from the technical and methodological consideration. Perhaps the most important of these is determination of right models of machine learning for different assignments like detection of objects (Deng *et al.*, 2021). Evaluating the architectural design which includes use of pre-trained models and transfer learning has been seen to yield massive improvements in the performance and

generalization of these models. Further, the concept of hyperparameter tuning was realised highly, especially in terms of balance between model sizes and accuracy.

The present research work has thus added value to the theoretical and practical reservoir on machine learning by showing how deep learning architectures offer a valid point of view in object detection. It also shed light on how machine learning can similarly change different fields including the autonomous driving, surveillance, and provide other intelligent systems of the future.

References

1. Adarsh, P., Rathi, P., & Kumar, M. 2020. YOLO v3-Tiny: Object Detection and Recognition using one stage improved model. *2020 6th International Conference on Advanced Computing and Communication Systems (ICACCS)*, 687-694.
2. Afdhal, A., Saddami, K., Sugiarto, S., Fuadi, Z. and Nasaruddin, N., 2023, August. Real-time object detection performance of yolov8 models for self-driving cars in a mixed traffic environment. In *2023 2nd International conferesnce on computer system, information technology, and electrical engineering (COSITE)* (pp. 260-265). IEEE.
3. Ahmad, I., & Pothuganti, K. 2020, August. Design & implementation of real time autonomous car by using image processing & IoT. In *2020 Third international conference on smart systems and inventive technology (ICSSIT)* (pp. 107-113). IEEE.
4. Alaba, S.Y. and Ball, J.E., 2023. Deep learning-based image 3-d object detection for autonomous driving. *IEEE Sensors Journal*, 23(4), pp.3378-3394.
5. Alkentar, S., Alsahwa, B., Assalem, A., & Karakolla, D. 2021. Practical comparison of the accuracy and speed of YOLO, SSD and Faster RCNN for drone detection. *Journal of Engineering*.
6. Bathla, G., Bhadane, K., Singh, R.K., Kumar, R., Aluvalu, R., Krishnamurthi, R., Kumar, A., Thakur, R.N. and Basheer, S., 2022. Autonomous vehicles and intelligent automation: Applications, challenges, and opportunities. *Mobile Information Systems*, 2022(1), p.7632892.
7. Bin Issa, R., Das, M., Rahman, M. S., Barua, M., Rhaman, M. K., Ripon, K. S. N., & Alam, M. G. R. 2021. Double deep Q-learning and faster R-Cnn-based autonomous vehicle navigation and obstacle avoidance in dynamic environment. *Sensors*, 21(4), 1468.
8. Boukerche, A., & Hou, Z. 2021. Object detection using deep learning methods in traffic scenarios. *ACM Computing Surveys (CSUR)*, 54(2), 1-35.
9. Cao, J., Song, C., Song, S., Xiao, F., Zhang, X., Liu, Z. and Ang Jr, M.H., 2021. Robust object tracking algorithm for autonomous vehicles in complex scenes. *Remote Sensing*, 13(16), p.3234.
10. Chen, C., Yao, G., Wang, C., Goudos, S. and Wan, S., 2022. Enhancing the robustness of object detection via 6G vehicular edge computing. *Digital Communications and Networks*, 8(6), pp.923-931.

11. Chen, C., Zhang, P., Zhang, H., Dai, J., Yi, Y., Zhang, H. and Zhang, Y., 2020. Deep learning on computational-resource-limited platforms: A survey. *Mobile Information Systems*, 2020(1), p.8454327.
12. Chen, G., 2024. Design of practical computer vision system with real-time object detection capability.
13. Chen, L., Lin, S., Lu, X., Cao, D., Wu, H., Guo, C., ... & Wang, F. Y. 2021. Deep neural network based vehicle and pedestrian detection for autonomous driving: A survey. *IEEE Transactions on Intelligent Transportation Systems*, 22(6), 3234-3246.
14. Chiu, Y. C., Tsai, C. Y., Ruan, M. D., Shen, G. Y., & Lee, T. T. 2020, August. Mobilenet-SSDv2: An improved object detection model for embedded systems. In *2020 International conference on system science and engineering (ICSSE)* (pp. 1-5). IEEE.
15. Chung, Q., Le, T., Dang, T., Vo, N., Nguyen, T., & Nguyen, K. 2020. Data Annotation Analysis in Vehicle Detection from Aerial Videos. *2020 RIVF International Conference on Computing and Communication Technologies (RIVF)*, 1-3.
16. Deng, Y., Zhang, T., Lou, G., Zheng, X., Jin, J., & Han, Q. L. 2021. Deep learning-based autonomous driving systems: A survey of attacks and defenses. *IEEE Transactions on Industrial Informatics*, 17(12), 7897-7912.
17. Fayyad, J., Jaradat, M.A., Gruyer, D. and Najjaran, H., 2020. Deep learning sensor fusion for autonomous vehicle perception and localization: A review. *Sensors*, 20(15), p.4220.
18. Feng, D., Haase-Schütz, C., Rosenbaum, L., Hertlein, H., Glaeser, C., Timm, F., ... & Dietmayer, K. 2020. Deep multi-modal object detection and semantic segmentation for autonomous driving: Datasets, methods, and challenges. *IEEE Transactions on Intelligent Transportation Systems*, 22(3), 1341-1360.
19. Formosa, N., Quddus, M., Ison, S., Abdel-Aty, M., & Yuan, J. 2020. Predicting real-time traffic conflicts using deep learning. *Accident Analysis & Prevention*, 136, 105429.
20. Geng, K., Dong, G. and Huang, W., 2022. Robust dual-modal image quality assessment aware deep learning network for traffic targets detection of autonomous vehicles. *Multimedia Tools and Applications*, 81(5), pp.6801-6826.
21. Gengec, N., Eker, O., Cevikalp, H., Yazici, A. and Yavuz, H.S., 2021. Visual object detection for autonomous transport vehicles in smart factories. *Turkish Journal of Electrical Engineering and Computer Sciences*, 29(4), pp.2101-2115.

22. Ghasemi, Y., Jeong, H., Choi, S. H., Park, K. B., & Lee, J. Y. 2022. Deep learning-based object detection in augmented reality: A systematic review. *Computers in Industry*, 139, 103661.
23. Giannaros, A., Karras, A., Theodorakopoulos, L., Karras, C., Kranias, P., Schizas, N., Kalogeratos, G. and Tsolis, D., 2023. Autonomous vehicles: Sophisticated attacks, safety issues, challenges, open topics, blockchain, and future directions. *Journal of Cybersecurity and Privacy*, 3(3), pp.493-543.
24. Gog, I., Kalra, S., Schafhalter, P., Wright, M.A., Gonzalez, J.E. and Stoica, I., 2021, May. Pylot: A modular platform for exploring latency-accuracy tradeoffs in autonomous vehicles. In *2021 IEEE International Conference on Robotics and Automation (ICRA)* (pp. 8806-8813). IEEE.
25. Grigorescu, S., Trasnea, B., Cocias, T., & Macesanu, G. 2020. A survey of deep learning techniques for autonomous driving. *Journal of field robotics*, 37(3), 362-386.
26. Gupta, A., Anpalagan, A., Guan, L. and Khwaja, A.S., 2021. Deep learning for object detection and scene perception in self-driving cars: Survey, challenges, and open issues. *Array*, 10, p.100057.
27. Haris, M., & Glowacz, A. 2021. RETRACTED: Road Object Detection: A Comparative Study of Deep Learning-Based Algorithms. *Electronics*, 10(16), 1932.
28. Hassaballah, M., Kenk, M.A., Muhammad, K. and Minaee, S., 2020. Vehicle detection and tracking in adverse weather using a deep learning framework. *IEEE transactions on intelligent transportation systems*, 22(7), pp.4230-4242.
29. Hoffmann, J. E., Tosso, H. G., Santos, M. M. D., Justo, J. F., Malik, A. W., & Rahman, A. U. 2020. Real-time adaptive object detection and tracking for autonomous vehicles. *IEEE Transactions on Intelligent Vehicles*, 6(3), 450-459.
30. Huang, Y., & Chen, Y. 2020. Autonomous driving with deep learning: A survey of state-of-art technologies. *arXiv preprint arXiv:2006.06091*.
31. Jha, S., Seo, C., Yang, E., & Joshi, G. P. 2021. Real time object detection and trackingsystem for video surveillance system. *Multimedia Tools and Applications*, 80(3), 3981-3996.

32. Jhong, S.Y., Chen, Y.Y., Hsia, C.H., Lin, S.C., Hsu, K.H. and Lai, C.F., 2021. Nighttime object detection system with lightweight deep network for internet of vehicles. *Journal of Real-Time Image Processing*, 18(4), pp.1141-1155.
33. Jiang, X., Yu, F. R., Song, T., Ma, Z., Song, Y., & Zhu, D. 2020. Blockchain-enabled cross-domain object detection for autonomous driving: A model sharing approach. *IEEE Internet of Things Journal*, 7(5), 3681-3692.
34. Juneja, A., Juneja, S., Soneja, A., & Jain, S. 2021. Real time object detection using CNN based single shot detector model. *Journal of Information Technology Management*, 13(1), 62-80.
35. Kaim, A., Cord, A.F. and Volk, M., 2018. A review of multi-criteria optimization techniques for agricultural land use allocation. *Environmental Modelling & Software*, 105, pp.79-93.
36. Kaur, P., Khehra, B., & Mavi, B. 2021. Data Annotation for Object Detection: A Review. *2021 IEEE International Midwest Symposium on Circuits and Systems (MWSCAS)*, 537-543.
37. Khosravian, A., Amirkhani, A., Kashiani, H. and Masih-Tehrani, M., 2021. Generalizing state-of-the-art object detectors for autonomous vehicles in unseen environments. *Expert Systems with Applications*, 183, p.115417.
38. Kim, J., Sung, J., & Park, S. 2020. Comparison of Faster-RCNN, YOLO, and SSD for Real-Time Vehicle Type Recognition. *2020 IEEE International Conference on Consumer Electronics - Asia (ICCE-Asia)*, 1-4.
39. Kuutti, S., Bowden, R., Jin, Y., Barber, P. and Fallah, S., 2020. A survey of deep learning applications to autonomous vehicle control. *IEEE Transactions on Intelligent Transportation Systems*, 22(2), pp.712-733.
40. Lee, D. H., & Liu, J. L. 2023. End-to-end deep learning of lane detection and path prediction for real-time autonomous driving. *Signal, Image and Video Processing*, 17(1), 199-205.
41. Lee, D. H., Chen, K. L., Liou, K. H., Liu, C. L., & Liu, J. L. 2021. Deep learning and control algorithms of direct perception for autonomous driving. *Applied Intelligence*, 51(1), 237-247.

42. Lee, H., Kang, S., & Chung, K. 2022. Robust Data Annotation Generative Adversarial Network for Object Detection. *Sensors (Basel, Switzerland)*, 23.
43. Li, Y., Shen, H., Fu, Y. and Wang, K., 2024. A method of dense point cloud SLAM based on improved YOLOV8 and fused with ORB-SLAM3 to cope with dynamic environments. *Expert Systems with Applications*, 255, p.124918.
44. Li, Y., Wang, H., Dang, L. M., Nguyen, T. N., Han, D., Lee, A., ... & Moon, H. 2020. A deep learning-based hybrid framework for object detection and recognition in autonomous driving. *IEEE Access*, 8, 194228-194239.
45. Liang, S., Wu, H., Zhen, L., Hua, Q., Garg, S., Kaddoum, G., ... & Yu, K. 2022. Edge YOLO: Real-time intelligent object detection system based on edge-cloud cooperation in autonomous vehicles. *IEEE Transactions on Intelligent Transportation Systems*, 23(12), 25345-25360.
46. Liu, R., Chen, Y., Wang, J., & Guo, Z. 2021. Attentive Mix: An Efficient Data Annotation Method for Object Detection. *2021 7th International Conference on Computer and Communications (ICCC)*, 770-774.
47. Mahaur, B., & Mishra, K. K. 2023. Small-object detection based on YOLOv5 in autonomous driving systems. *Pattern Recognition Letters*, 168, 115-122.
48. Mahaur, B., Singh, N. and Mishra, K.K., 2022. Road object detection: a comparative study of deep learning-based algorithms. *Multimedia Tools and Applications*, 81(10), pp.14247-14282.
49. Mauri, A., Khemmar, R., Decoux, B., Ragot, N., Rossi, R., Trabelsi, R., ... & Savatier, X. 2020. Deep learning for real-time 3D multi-object detection, localisation, and tracking: Application to smart mobility. *Sensors*, 20(2), 532.
50. Muhammad, K., Ullah, A., Lloret, J., Del Ser, J. and de Albuquerque, V.H.C., 2020. Deep learning for safe autonomous driving: Current challenges and future directions. *IEEE Transactions on Intelligent Transportation Systems*, 22(7), pp.4316-4336.
51. Ni, J., Chen, Y., Chen, Y., Zhu, J., Ali, D., & Cao, W. 2020. A survey on theories and applications for self-driving cars based on deep learning methods. *Applied Sciences*, 10(8), 2749.

52. Ni, J., Shen, K., Chen, Y., Cao, W., & Yang, S. X. 2022. An improved deep network-based scene classification method for self-driving cars. *IEEE Transactions on Instrumentation and Measurement*, 71, 1-14.
53. Niranjana, D., Vinaykarthik, B., & , M. 2021. Performance Analysis of SSD and Faster RCNN Multi-class Object Detection Model for Autonomous Driving Vehicle Research Using CARLA Simulator. 2021 Fourth International Conference on Electrical, Computer and Communication Technologies (ICECCT), 1-6.
54. Olamofe, J., Dong, X., Qian, L., & Shields, E. 2022. Performance Evaluation of Data Annotation for Object Detection in XView Dataset. 2022 *International Conference on Intelligent Data Science Technologies and Applications (IDSTA)*, 26-33.
55. Pal, S. K., Pramanik, A., Maiti, J., & Mitra, P. 2021. Deep learning in multi-object detection and tracking: state of the art. *Applied Intelligence*, 51, 6400-6429.
56. Parekh, D., Poddar, N., Rajpurkar, A., Chahal, M., Kumar, N., Joshi, G.P. and Cho, W., 2022. A review on autonomous vehicles: Progress, methods and challenges. *Electronics*, 11(14), p.2162.
57. Pavel, M.I., Tan, S.Y. and Abdullah, A., 2022. Vision-based autonomous vehicle systems based on deep learning: A systematic literature review. *Applied Sciences*, 12(14), p.6831.
58. Rahmaniar, W., & Hernawan, A. 2021. Real-time human detection using deep learning on embedded platforms: A review. *Journal of Robotics and Control (JRC)*, 2(6), 462-468.
59. Razi, A., Chen, X., Li, H., Wang, H., Russo, B., Chen, Y. and Yu, H., 2023. Deep learning serves traffic safety analysis: A forward-looking review. *IET Intelligent Transport Systems*, 17(1), pp.22-71.
60. Shahian Jahromi, B., Tulabandhula, T. and Cetin, S., 2019. Real-time hybrid multi-sensor fusion framework for perception in autonomous vehicles. *Sensors*, 19(20), p.4357.
61. Sharma, P. and Rana, C., 2024. Artificial intelligence based object detection and traffic prediction by autonomous vehicles—A review. *Expert Systems with Applications*, p.124664.
62. Sharma, T., Debaque, B., Duclos, N., Chehri, A., Kinder, B., & Fortier, P. 2022. Deep learning-based object detection and scene perception under bad weather conditions. *Electronics*, 11(4), 563.

63. Song, Z., Liu, L., Jia, F., Luo, Y., Jia, C., Zhang, G., Yang, L. and Wang, L., 2024. Robustness-aware 3d object detection in autonomous driving: A review and outlook. *IEEE Transactions on Intelligent Transportation Systems*.
64. Srivast, S., Divekar, A., Anilkumar, C., Naik, I., Kulkarni, V., & V., P. 2020. Comparative analysis of deep learning image detection algorithms. *Journal of Big Data*, 8, 1-27.
65. Staroverov, A., Yudin, D.A., Belkin, I., Adeshkin, V., Solomentsev, Y.K. and Panov, A.I., 2020. Real-time object navigation with deep neural networks and hierarchical reinforcement learning. *IEEE Access*, 8, pp.195608-195621.
66. Tan, L., Huangfu, T., Wu, L., & Chen, W. 2021. Comparison of YOLO v3, Faster R-CNN, and SSD for Real-Time Pill Identification.
67. Wang, H., Liu, C., Cai, Y., Chen, L. and Li, Y., 2024. YOLOv8-QSD: An improved small object detection algorithm for autonomous vehicles based on YOLOv8. *IEEE Transactions on Instrumentation and Measurement*.
68. Wang, H., Yu, Y., Cai, Y., Chen, X., Chen, L. and Liu, Q., 2019. A comparative study of state-of-the-art deep learning algorithms for vehicle detection. *IEEE Intelligent Transportation Systems Magazine*, 11(2), pp.82-95.
69. Wu, D., Liao, M. W., Zhang, W. T., Wang, X. G., Bai, X., Cheng, W. Q., & Liu, W. Y. 2022. Yolop: You only look once for panoptic driving perception. *Machine Intelligence Research*, 19(6), 550-562.
70. Yang, J., Liu, S., Li, Z., Li, X. and Sun, J., 2022. Real-time object detection for streaming perception. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 5385-5395).
71. Yang, K., Tang, X., Li, J., Wang, H., Zhong, G., Chen, J. and Cao, D., 2023. Uncertainties in onboard algorithms for autonomous vehicles: Challenges, mitigation, and perspectives. *IEEE Transactions on Intelligent Transportation Systems*, 24(9), pp.8963-8987.
72. Yang, L., Zhong, J., Zhang, Y., Bai, S., Li, G., Yang, Y., & Zhang, J. 2023. An Improving Faster-RCNN With Multi-Attention ResNet for Small Target Detection in Intelligent Autonomous Transport With 6G. *IEEE Transactions on Intelligent Transportation Systems*, 24, 7717-7725.

73. Younis, A., Shixin, L., Jn, S., & Hai, Z. 2020, January. Real-time object detection using pre-trained deep learning models MobileNet-SSD. In *Proceedings of 2020 6th International Conference on Computing and Data Engineering* (pp. 44-48).
74. Zamanakos, G., Tsochatzidis, L., Amanatiadis, A., & Pratikakis, I. 2021. A comprehensive survey of LIDAR-based 3D object detection methods with deep learning for autonomous driving. *Computers & Graphics*, 99, 153-181.
75. Zhao, Z.Q., Zheng, P., Xu, S.T. and Wu, X., 2019. Object detection with deep learning: A review. *IEEE transactions on neural networks and learning systems*, 30(11), pp.3212-3232.

Appendix 1

Dataset Link

content/drive/MyDrive/Dissertation/data

Code

REAL-TIME OBJECT DETECTION FOR AUTONOMOUS VEHICLES USING DEEP LEARNING

[+ Code](#)[+ Text](#)

✓ 1 - Data Set

- Mounting Drive
- Access Dataset

```
[ ] # Mount Google Drive to access the dataset
    from google.colab import drive
    drive.mount('/content/drive')
```

 Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```
[ ] extracted_dir = '/content/drive/MyDrive/Dissertation'
```

✓ 1.1 - Removing the corrupted images from the dataset.

```

from PIL import UnidentifiedImageError
from PIL import Image
import os

extracted_dir = '/content/drive/MyDrive/Dissertation/data'
# Function to check for corrupted images
def remove_corrupted_images(directory):
    num_corrupted = 0
    for subdir, __, files in os.walk(directory):
        for file in files:
            img_path = os.path.join(subdir, file)
            try:
                img = Image.open(img_path)
                img.verify() # Check if image is corrupted
            except (IOError, UnidentifiedImageError):
                print(f"Corrupted image found and removed: {img_path}")
                os.remove(img_path)
                num_corrupted += 1
    print(f"Total corrupted images removed: {num_corrupted}")

# Run the function on your dataset
remove_corrupted_images(extracted_dir)

```

➡ Total corrupted images removed: 0

✓ 1.2 - Resizing the images(Dataset)

```

[ ] def resize_images(directory, target_size=(512, 512)):
    for subdir, __, files in os.walk(directory):
        for file in files:
            img_path = os.path.join(subdir, file)
            img = Image.open(img_path)
            img_resized = img.resize(target_size)
            img_resized.save(img_path)

# Resize all images
resize_images(extracted_dir, target_size=(512, 512))

```

✓ 2 - Data Pre-process

```
[ ] import numpy as np

def preprocess_images(directory):
    for subdir, __, files in os.walk(directory):
        for file in files:
            img_path = os.path.join(subdir, file)
            img = Image.open(img_path)
            img_array = np.asarray(img) / 255.0
            img_preprocessed = Image.fromarray((img_array * 255).astype(np.uint8))
            img_preprocessed.save(img_path)

# Preprocess images
preprocess_images(extracted_dir)
```

✓ 2.1 - Displaying Images

```
[ ] import matplotlib.pyplot as plt
import random
import os
from PIL import Image

def display_sample_images(directory, num_samples=5):
    img_paths = []

    # Collect all image file paths from the directory
    for subdir, __, files in os.walk(directory):
        for file in files:
            if file.endswith((''.png', '.jpg', '.jpeg', '.bmp', '.gif')): # Ensure valid image extensions
                img_paths.append(os.path.join(subdir, file))

    # Ensure num_samples does not exceed the number of available images
    if len(img_paths) == 0:
        raise ValueError("No images found in the directory.")
    num_samples = min(num_samples, len(img_paths)) # Adjust if necessary

    # Randomly sample images from the list
    sample_images = random.sample(img_paths, num_samples)

    # Plot the images
    plt.figure(figsize=(10, 10))
    for i, img_path in enumerate(sample_images):
        img = Image.open(img_path)
        plt.subplot(1, num_samples, i + 1)
        plt.imshow(img)
        plt.title(os.path.basename(img_path))
        plt.axis('off')

    plt.show()
```

2.2 - Importing YOLOv5

```
# clone yolov5 model
!git clone https://github.com/ultralytics/yolov5
%cd yolov5

!pip install -r requirements.txt
```

```
[ ] import torch

model = torch.hub.load('ultralytics/yolov5', 'yolov5s')
```

/usr/local/lib/python3.10/dist-packages/torch/hub.py:295: UserWarning: You are about to download and run code from an untrusted repository. In a future release, this won't be allowed. To warnings.warn(
Downloading: "https://github.com/ultralytics/yolov5/zipball/master" to /root/.cache/torch/hub/master.zip
Creating new Ultralytics Settings v0.0.6 file 
View Ultralytics Settings with 'yolo settings' or at '/root/.config/Ultralytics/settings.json'
Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dir=path/to/dir'. For help see <https://docs.ultralytics.com/quickstart/#Ultralytics-settings>.
YOLOv5  2024-9-24 Python-3.10.12 torch-2.4.1+cu121 CUDA:0 (Tesla T4, 15102MiB)
Downloading <https://github.com/ultralytics/yolov5/releases/download/v7.0/yolov5s.pt> to yolov5s.pt...
100%  14.1M/14.1M [00:00<00:00, 450MB/s]
Fusing layers...
YOLOv5s summary: 213 layers, 7225885 parameters, 0 gradients, 16.4 GFLOPs
Adding AutoShape...

2.3 - Importing Directories & Subfolder

```
import os
from pathlib import Path

# Define the input and output directories
input_dir = '/content/drive/MyDrive/Dissertation/data'
output_dir = '/content/drive/MyDrive/UKA362/Dissertation/AnnotatedImages'

# Create output directory if it doesn't exist
if not os.path.exists(output_dir):
    os.makedirs(output_dir)

# List all the subfolders (categories) inside the Dataset directory
subfolders = [f.path for f in os.scandir(input_dir) if f.is_dir()]
```

```
for subfolder in subfolders:
    output_subfolder = os.path.join(output_dir, os.path.basename(subfolder))
    if not os.path.exists(output_subfolder):
        os.makedirs(output_subfolder)

    image_paths = [str(p) for p in Path(subfolder).rglob('*.*') if p.suffix in ['.png', '.jpg']]

    if image_paths:
        results = model(image_paths)

        results.save(save_dir=output_subfolder)
    else:
        print(f"No images found in {subfolder}")
```

▼ 2.4 - Cleaning and re-naming folder

```
[ ] import shutil

def clean_and_rename_folders(base_dir):
    for folder_name in os.listdir(base_dir):
        folder_path = os.path.join(base_dir, folder_name)

        if os.path.isdir(folder_path):
            if folder_name.endswith('2'):
                base_name = folder_name[:-1]
                base_path = os.path.join(base_dir, base_name)
                if not os.path.exists(base_path):
                    os.makedirs(base_path)
                for file_name in os.listdir(folder_path):
                    shutil.move(os.path.join(folder_path, file_name), os.path.join(base_path, file_name))
                os.rmdir(folder_path)

clean_and_rename_folders('/content/drive/MyDrive/UKA362/Dissertation/AnnotatedImages')
```

▼ 2.5 - Splitting data into test and train

```
import os

# Define paths
images_dir = '/content/drive/MyDrive/UKA362/Dissertation/data'
annotations_dir = '/content/drive/MyDrive/UKA362/Dissertation/AnnotatedImages'
train_images_dir = '/content/drive/MyDrive/UKA362/Dissertation/TrainImages'
train_annotations_dir = '/content/drive/MyDrive/UKA362/Dissertation/TrainAnnotations'
test_images_dir = '/content/drive/MyDrive/UKA362/Dissertation/TestImages'
test_annotations_dir = '/content/drive/MyDrive/UKA362/Dissertation/TestAnnotations'

# Create directories for training and testing sets
for subfolder in os.listdir(images_dir):
    subfolder_path_images = os.path.join(images_dir, subfolder)
    subfolder_path_annotations = os.path.join(annotations_dir, subfolder)

    if os.path.isdir(subfolder_path_images):
        os.makedirs(os.path.join(train_images_dir, subfolder), exist_ok=True)
        os.makedirs(os.path.join(test_images_dir, subfolder), exist_ok=True)
        os.makedirs(os.path.join(train_annotations_dir, subfolder), exist_ok=True)
        os.makedirs(os.path.join(test_annotations_dir, subfolder), exist_ok=True)
```

```

import shutil
import random

def split_dataset(src_images_dir, src_annotations_dir, dest_images_train_dir, dest_annotations_train_dir, dest_images_test_dir, dest_annotations_test_dir, test_size=0.2):
    for category in os.listdir(src_images_dir):
        category_images_path = os.path.join(src_images_dir, category)
        category_annotations_path = os.path.join(src_annotations_dir, category)

        if os.path.isdir(category_images_path):
            # List all image files
            image_files = [f for f in os.listdir(category_images_path) if f.endswith(('.png', '.jpg'))]
            num_images = len(image_files)
            num_test = int(num_images * test_size)

            # Shuffle and split
            random.shuffle(image_files)
            test_files = image_files[:num_test]
            train_files = image_files[num_test:]

            # Move image files to train and test directories
            for file in train_files:
                shutil.move(os.path.join(category_images_path, file), os.path.join(dest_images_train_dir, category, file))
                # Move corresponding annotation file
                annotation_file = file.replace('.png', '.jpg') # Change based on your file format
                shutil.move(os.path.join(category_annotations_path, annotation_file), os.path.join(dest_annotations_train_dir, category, annotation_file))

            for file in test_files:
                shutil.move(os.path.join(category_images_path, file), os.path.join(dest_images_test_dir, category, file))
                # Move corresponding annotation file
                annotation_file = file.replace('.png', '.jpg') # Change based on your file format
                shutil.move(os.path.join(category_annotations_path, annotation_file), os.path.join(dest_annotations_test_dir, category, annotation_file))

    split_dataset(images_dir, annotations_dir, train_images_dir, train_annotations_dir, test_images_dir, test_annotations_dir)

```

2.6 - Annotation of the images

```
[ ]
import cv2
import json
import os
import numpy as np

# Define directories
train_annotation_dir = '/content/drive/MyDrive/UCAS2/annotation/train/annotations'
test_annotation_dir = '/content/drive/MyDrive/UCAS2/annotation/test/annotations'
train_img_dir = '/content/train'
test_img_dir = '/content/test'

# Create new directories if they don't exist
os.makedirs(train_img_dir, exist_ok=True)
os.makedirs(test_img_dir, exist_ok=True)

def convert_annotation_to_json(img_path, json_path):
    # Read the annotation image
    img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE) # Read as grayscale

    # Convert image to binary format or extract required information
    annotations = []
    unique_values = np.unique(img)
    for value in unique_values:
        if value == 0: # Assuming 0 is background
            continue
        # Find contours for each class
        contours, _ = cv2.findContours(img == value, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
        for contour in contours:
            contour = contour.reshape(-1, 2)
            x, y, w, h = cv2.boundingRect(contour)
            annotations.append({
                'class': int(value), # class label or name
                'box': [x, y, w, h] # [x, y, width, height]
            })

    # Save annotations to json file
    with open(json_path, 'w') as f:
        json.dump(annotations, f, indent=4)

def process_annotations(source_dir, target_dir):
    for subfolder in os.listdir(source_dir):
        subfolder_path = os.path.join(source_dir, subfolder)
        json_subfolder = os.path.join(target_dir, subfolder)
        os.makedirs(json_subfolder, exist_ok=True)

        for file_name in os.listdir(subfolder_path):
            if file_name.endswith('.jpg'):
                img_path = os.path.join(subfolder_path, file_name)
                json_file_name = file_name.replace('.jpg', '.json')
                json_path = os.path.join(json_subfolder, json_file_name)
                convert_annotation_to_json(img_path, json_path)

# Process train and test annotations
process_annotations(train_annotation_dir, train_img_dir)
process_annotations(test_annotation_dir, test_img_dir)

print('Annotation conversion completed.')
```

3.0 - Importing Library and API

```
[ ] !pip install tensorflow
!pip install tensorflow-addons
!pip install tf_cli
!pip install tf-models-official

[ ] # Navigate to your working directory and download the model
!wget http://download.tensorflow.org/models/object_detection/tf2/20200711/efficientdet_d0_coco17_tpu-32.tar.gz
!tar -xvf efficientdet_d0_coco17_tpu-32.tar.gz

[ ] # Step 1: Update pip, setuptools, and wheel
!pip install --upgrade pip setuptools wheel
# Step 2: Install necessary dependencies
!pip install Cython contextlib2 pillow lxml jupyter matplotlib opencv-python-headless tensorflow-addons

# Step 3: Install Protobuf Compiler (Linux & macOS)
# For Ubuntu
!sudo apt-get install protobuf-compiler
# For macOS
!brew install protobuf

# Check Protobuf installation
!protoc --version

# Step 4: Clone TensorFlow Models and set up Object Detection API
!git clone https://github.com/tensorflow/models.git
!cd models/research

# Step 5: Compile Protobuf files
!protoc object_detection/protos/*.proto --python_out=.

# Step 6: Add the models/research and models/research/slim directories to PYTHONPATH
!export PYTHONPATH=$PYTHONPATH:`pwd`:`pwd`/slim

# Step 7: Install TensorFlow Object Detection API
!python -m pip install .

# Step 8: Install TensorFlow (in case it's not installed already or needs upgrading)
!pip install --upgrade tensorflow

# (Optional) Verify TensorFlow installation
!python -c "import tensorflow as tf; print(tf.__version__)"
```

```
[ ] # Clone the TensorFlow models repository
!git clone https://github.com/tensorflow/models.git
# Navigate to the models/research directory
!cd models/research/

# Install Protobuf compiler
!apt-get install -y protobuf-compiler

# Compile the protobuf files
!protoc object_detection/protos/*.proto --python_out=.

# Install TensorFlow Object Detection API dependencies
!pip install .

# Test if the installation was successful
!python object_detection/builders/model_builder_tf2_test.py
!pip install tensorflow
!import tensorflow as tf
!print(tf.__version__)

[ ] # Install TensorFlow and the required Object Detection API
!pip install tensorflow
!pip install tf_cli

# Clone the TensorFlow models repository to get the object_detection package
!git clone https://github.com/tensorflow/models.git

# Navigate to the research directory
!cd models/research

# Install the Object Detection API
!protoc object_detection/protos/*.proto --python_out=.
!pip install .
```

3.1 - Feature extraction

```

import tensorflow as tf
import os

# Utility functions to create features
def bytes_feature(value):
    """Returns a bytes_list from a string / byte."""
    return tf.train.Feature(bytes_list=tf.train.BytesList(value=[value]))

def int64_feature(value):
    """Returns an int64_list from a bool / enum / int / uint."""
    return tf.train.Feature(int64_list=tf.train.Int64List(value=[value]))

def float_list_feature(value):
    """Returns a float_list from a float / double."""
    return tf.train.Feature(float_list=tf.train.FloatList(value=value))

def create_tf_example(image_path, annotations):
    """
    Creates a single TFExample from image path and its corresponding annotations.
    """
    with tf.io.gfile.GFile(image_path, 'rb') as fid:
        encoded_image_data = fid.read()

    # Extract image metadata
    image = tf.image.decode_jpeg(encoded_image_data)
    height = image.shape[0]
    width = image.shape[1]

    # Create a TFRecord Example object
    tf_example = tf.train.Example(features=tf.train.Features(feature={
        'image/height': int64_feature(height),
        'image/width': int64_feature(width),
        'image/encoded': bytes_feature(encoded_image_data),
        'image/format': bytes_feature(b'jpeg'),
        # Add bbox and other fields based on annotations here...
    }))
    return tf_example

def write_tfrecord(dataset_dir, output_path):
    """
    Write the dataset into a TFRecord file.
    """
    writer = tf.io.TFRecordWriter(output_path)

    for image_filename in os.listdir(dataset_dir):
        if image_filename.endswith(".jpg"):
            image_path = os.path.join(dataset_dir, image_filename)
            # Assuming annotation info comes from your pre-existing labels
            annotation = {} # Load corresponding annotation (adjust accordingly)
            tf_example = create_tf_example(image_path, annotation)
            writer.write(tf_example.SerializeToString())
    writer.close()

```

4.0 - Modelling

✓ 1. R-CNN

```
[ ] import torch
from torch.utils.data import Dataset
import torchvision
from torchvision.transforms import functional as F
from torchvision.models.detection.faster_rcnn import FastRCNNPredictor
from PIL import Image
import json
import os

# Custom Dataset class to handle subdirectories
class CustomObjectDetectionDataset(Dataset):
    def __init__(self, images_dir, annotations_dir, transforms=None):
        self.images_dir = images_dir
        self.annotations_dir = annotations_dir
        self.transforms = transforms
        self.image_paths = []
        self.annotation_paths = []

        # Recursively find all image files and their corresponding annotation files
        for root, dirs, files in os.walk(images_dir):
            for file in files:
                if file.endswith('.jpg'):
                    img_path = os.path.join(root, file)
                    annotation_filename = file.replace('.jpg', '.json')
                    annotation_path = os.path.join(annotations_dir, os.path.basename(root), annotation_filename)

                    if os.path.exists(annotation_path):
                        self.image_paths.append(img_path)
                        self.annotation_paths.append(annotation_path)

    def __len__(self):
        return len(self.image_paths)

    def __getitem__(self, idx):
        image_path = self.image_paths[idx]
        annotation_path = self.annotation_paths[idx]

        # Load image
        image = Image.open(image_path).convert("RGB")

        # Load annotations
        with open(annotation_path) as f:
            annotations = json.load(f)

        # Convert annotations to the format expected by Faster R-CNN
        boxes = []
        labels = []
```

```

# Define data loaders
train_loader = torch.utils.data.DataLoader(train_dataset, batch_size=2, shuffle=True, collate_fn=lambda x: tuple(zip(*x)))
test_loader = torch.utils.data.DataLoader(test_dataset, batch_size=2, shuffle=False, collate_fn=lambda x: tuple(zip(*x)))

# Get the unique classes from the training dataset
all_classes = get_all_classes(train_dataset)
num_classes = len(all_classes) + 1

print(f"Number of classes (including background): {num_classes}")

# Load the pre-trained Faster R-CNN model
model = torchvision.models.detection.FasterRCNN_ResNet50_FPN(pretrained=True)

# Get the number of input features for the classifier (from the box_predictor)
in_features = model.roi_heads.box_predictor.cls_score.in_features

# Define the number of classes (including background)
num_classes = len(all_classes) + 1 # +1 for background

# Replace the head of the model with a new one that has the correct number of classes
model.roi_heads.box_predictor = FastRCNNPredictor(in_features, num_classes)

# Define optimizer and early stopping parameters
params = [p for p in model.parameters() if p.requires_grad]
optimizer = torch.optim.SGD(params, lr=0.005, momentum=0.9, weight_decay=0.0005)

# Early stopping parameters
best_loss = np.inf
patience = 5
counter = 0

import torchvision.transforms.functional as F

# Training loop with early stopping
num_epochs = 20
early_stopping = False
for epoch in range(num_epochs):
    if early_stopping:
        break

    model.train()
    running_loss = 0.0
    for images, targets in train_loader:
        # Convert PIL images to torch tensors
        images = [F.to_tensor(image) for image in images]

        targets = [{"k": v for k, v in t.items()} for t in targets]

        optimizer.zero_grad()
        loss_dict = model(images, targets)
        losses = sum(loss for loss in loss_dict.values())
        losses.backward()
        optimizer.step()
        running_loss += losses.item()

    avg_loss = running_loss / len(train_loader)
    print(f"Epoch {epoch+1}/{num_epochs}, Loss: {avg_loss}")

```

R-CNN Evaluation

```
[ ] from torchvision.ops import box_iou

# Function to calculate Intersection over Union (IoU)
def calculate_iou(pred_boxes, true_boxes):
    ious = []
    for i, pred_box in enumerate(pred_boxes):
        iou = box_iou(pred_box.unsqueeze(0), true_boxes).item()
        ious.append(iou)
    return ious

# Function to evaluate the model
def evaluate_model(model, data_loader):
    model.eval()
    iou_scores = []
    with torch.no_grad():
        for images, targets in data_loader:
            images = [F.to_tensor(image) for image in images]
            predictions = model(images)

            for i, prediction in enumerate(predictions):
                pred_boxes = prediction['boxes']
                true_boxes = targets[i]['boxes']

                # Calculate IoU for each prediction
                ious = calculate_iou(pred_boxes, true_boxes)
                iou_scores.extend(ious)

    avg_iou = sum(iou_scores) / len(iou_scores) if iou_scores else 0
    print(f"Average IoU: {avg_iou:.4f}")

# Evaluate on the test set
evaluate_model(model, test_loader)
```

2. RetinaNet

```

import torch
from torch.utils.data import Dataset
import torchvision
from torchvision.transforms import functional as F
from torchvision.models.detection.retinanet import RetinaNet, RetinaNetHead
from PIL import Image
import json
import os

# Custom Dataset class
class CustomObjectDetectionDataset(Dataset):
    def __init__(self, images_dir, annotations_dir, transforms=None):
        self.images_dir = images_dir
        self.annotations_dir = annotations_dir
        self.transforms = transforms
        self.image_paths = []
        self.annotation_paths = []

        # Recursively find all image files and their corresponding annotation files
        for root, dirs, files in os.walk(images_dir):
            for file in files:
                if file.endswith('.jpg'):
                    img_path = os.path.join(root, file)
                    annotation_filename = file.replace('.jpg', '.json')
                    annotation_path = os.path.join(annotations_dir, os.path.basename(root), annotation_filename)

                    if os.path.exists(annotation_path):
                        self.image_paths.append(img_path)
                        self.annotation_paths.append(annotation_path)

    def __len__(self):
        return len(self.image_paths)

    def __getitem__(self, idx):
        image_path = self.image_paths[idx]
        annotation_path = self.annotation_paths[idx]

        # Load image
        image = Image.open(image_path).convert("RGB")

        # Load annotations
        with open(annotation_path) as f:
            annotations = json.load(f)

        # Convert annotations to the format expected by RetinaNet
        boxes = []
        labels = []
        for ann in annotations:
            x, y, w, h = ann['bbox']
            boxes.append([x, y, x+w, y+h])
            labels.append(ann['class'])

        boxes = torch.as_tensor(boxes, dtype=torch.float32)
        labels = torch.as_tensor(labels, dtype=torch.int64)

```

3. EfficientDet

[] !pip install timm torchvision

```
import os
import json
import torch
from torch.utils.data import Dataset
from PIL import Image
import torchvision.transforms.functional as F

class CustomObjectDetectionDataset(Dataset):
    def __init__(self, images_dir, annotations_dir, transforms=None):
        self.images_dir = images_dir
        self.annotations_dir = annotations_dir
        self.transforms = transforms
        self.image_paths = []
        self.annotation_paths = []

        # Traverse directories and find all images and their corresponding annotations
        for subfolder in os.listdir(images_dir):
            subfolder_path = os.path.join(images_dir, subfolder)
            if not os.path.isdir(subfolder_path):
                continue

            for file in os.listdir(subfolder_path):
                if file.endswith('.png'):
                    img_path = os.path.join(subfolder_path, file)
                    annotation_filename = file.replace('.png', '.json')
                    annotation_path = os.path.join(annotations_dir, subfolder, annotation_filename)

                    if os.path.exists(annotation_path):
                        self.image_paths.append(img_path)
                        self.annotation_paths.append(annotation_path)

    def __len__(self):
        return len(self.image_paths)

    def __getitem__(self, idx):
        image_path = self.image_paths[idx]
        annotation_path = self.annotation_paths[idx]

        # Load image
        image = Image.open(image_path).convert("RGB")

        # Load annotations
        with open(annotation_path) as f:
            annotations = json.load(f)

        # Convert annotations to the format expected by EfficientDet
        boxes = []
        labels = []
        for ann in annotations:
            x, y, w, h = ann["bbox"]
            boxes.append([x, y, x+w, y+h])
```

```

targets = [(k: v for k, v in t.items()) for t in targets]

optimizer.zero_grad()
loss_dict = model(images, targets)
losses = sum(loss for loss in loss_dict.values())
losses.backward()
optimizer.step()
running_loss += losses.item()

avg_loss = running_loss / len(train_loader)
print(f"Epoch {epoch+1}/{num_epochs}, Loss: {avg_loss}")

# Early stopping
if avg_loss < best_loss:
    best_loss = avg_loss
    counter = 0
    # Save best model
    torch.save(model.state_dict(), 'best_efficientdet_model.pth')
else:
    counter += 1
    if counter >= patience:
        print("Early stopping triggered!")
        early_stopping = True

# Load the best model
model.load_state_dict(torch.load('best_efficientdet_model.pth'))
model.eval()

# Function to calculate Intersection over Union (IoU)
def calculate_iou(pred_boxes, true_boxes):
    ious = []
    for i, pred_box in enumerate(pred_boxes):
        iou = box_iou(pred_box.unsqueeze(0), true_boxes).item()
        ious.append(iou)
    return ious

# Function to evaluate the model
def evaluate_model(model, data_loader):
    model.eval()
    iou_scores = []
    with torch.no_grad():
        for images, targets in data_loader:
            images = [F.to_tensor(image) for image in images]
            predictions = model(images)

            for i, prediction in enumerate(predictions):
                pred_boxes = prediction['boxes']
                true_boxes = targets[i]['boxes']

                # Calculate IoU for each prediction
                ious = calculate_iou(pred_boxes, true_boxes)
                iou_scores.extend(ious)

    avg_iou = sum(iou_scores) / len(iou_scores) if iou_scores else 0
    print(f"Average IoU: {avg_iou:.4f}")

# Evaluate on the test set
evaluate_model(model, test_loader)

```

Appendix 2

Letter of confirmation



College of Engineering, Design and Physical Sciences Research Ethics Committee
Brunel University London
Kingston Lane
Uxbridge
UB8 3PH
United Kingdom
www.brunel.ac.uk

16 August 2024

LETTER OF CONFIRMATION

Applicant: Mr Akshay Pisal

Project Title: Deep Learning Models for Real-time Object Detection for Self-driving Cars

Reference: 49457-NER-Aug/2024- 52626-1

Dear Mr Akshay Pisal

The Research Ethics Committee has considered the above application recently submitted by you.

This letter is to confirm that, according to the information provided in your BREO application, your project does not require full ethical review. You may proceed with your research as set out in your submitted BREO application, using secondary data sources only. You may not use any data sources for which you have not sought approval.

Please note that:

- You are not permitted to conduct research involving human participants, their tissue and/or their data. If you wish to conduct such research (including surveys, questionnaires, interviews etc.), you must contact the Research Ethics Committee to seek approval prior to engaging with any participants or working with data for which you do not have approval.
- The Research Ethics Committee reserves the right to sample and review documentation relevant to the study.
- If during the course of the study, you would like to carry out research activities that concern a human participant, their tissue and/or their data, you must submit a new BREO application and await approval before proceeding. Research activity includes the recruitment of participants, undertaking consent procedures and collection of data. Breach of this requirement constitutes research misconduct and is a disciplinary offence.

Good luck with your research!

Kind regards,

Professor Simon Taylor

Chair of the College of Engineering, Design and Physical Sciences Research Ethics Committee

Brunel University London